openSecureKey — eigener Sicherheitsschlüssel mit Passwortspeicher

Zusammenhang

Es gibt keinen Schlüssel, der alles zusammen kann: FIDO2/Passkeys, Smartcard mit eigenen Zertifikaten, 2FA-Codes **und** einen echten Passwortspeicher, abrufbar per USB, NFC und Bluetooth. Nitrokey und YubiKey lassen jeweils ein Stück davon weg. Ziel ist ein eigenes Gerät: Schaltplan und Platine selbst gezeichnet, Firmware und Verwaltungsprogramm selbst geschrieben, Schlüssel in einem eigenen Sicherheitschip.

Entschieden in diesem Gespräch:

- **nRF52840** als Rechenkern (macht USB, BLE und NFC in einem Chip)
- **NXP SE050** als Sicherheitschip am I2C — Schlüssel verlassen ihn nie
- Start auf **Steckbrett/Entwicklungsboard**, Ziel ist die eigene Platine
- Programm-Reihenfolge: **Windows** → **Linux** → **iPhone** → **Mac** → **Android**
- **Display**: kleines OLED, das den Vorgang selbst anzeigt
- **Fingerabdrucksensor** als dritter Faktor, von Anfang an im Schaltplan
- **Mit Akku** (geändert): LiPo-Zelle ~40–100 mAh, lädt am USB. Nötig, weil der nRF52840 sich nicht aus dem NFC-Feld versorgen kann. Damit gehen NFC und Bluetooth auch ohne Rechner, und die **Uhr läuft weiter** — Voraussetzung für TOTP und für Zeitstempel bei Signaturen. Bauform: Schlüsselanhänger mit USB-C-Stecker, Laderegler und Akkuschutz auf der Platine.
- **Die Software liegt auf dem Stick selbst**: eine kleine, schreibgeschützte Laufwerks-Partition mit den portablen Programmen für Windows, Mac und Linux. Einstecken, starten, keine Installation, **keine Adminrechte**.

Was "von jeder App nutzbar" wirklich heißt

Drei Teile sind echte Standards und brauchen keinen Treiber und keine App:

- **FIDO2/WebAuthn/Passkeys** über USB-HID — jeder Browser, Windows Hello, SSH
- **Smartcard (PIV + OpenPGP)** über USB-CCID — darüber laufen S/MIME, Code-Signatur, SSH-Schlüssel, VPN, Windows-Anmeldung, Dokumente signieren
- **TOTP (RFC 6238)** — dieselben Codes wie Google Authenticator

Einen Standard, über den jede App Passwörter aus einem Stick holt, gibt es **nicht**. Darum bauen wir alle Wege, die es gibt — jede Software findet einen davon:

Weg	erreicht damit
Gerät tippt als USB-Tastatur	jede Software, auch Anmeldebildschirm und BIOS
PKCS#11-Bibliothek (dll/.so/.dylib)	Firefox, Thunderbird, OpenSSL, OpenSSH, Java, VPN-Clients, Signierwerkzeuge
Smartcard-Minidriver + CNG (Windows)	jede Windows-App über den Zertifikatsspeicher, Windows-Anmeldung
CryptoTokenKit (Mac/iPhone)	Safari, Mail, Keychain-Apps
ssh-agent (unser Programm bietet einen an)	`ssh`, `git`, `scp`, Ansible, alles was SSH nutzt
FIDO-SSH (`sk-ecdsa`)	OpenSSH ohne jedes Zusatzprogramm
Browser-Erweiterung	Anmeldefelder ausfüllen in Chrome, Firefox, Edge
Kommandozeile `osk get <name>`	Skripte, CI, Umgebungsvariablen, Docker
lokale Schnittstelle (Loopback/Named Pipe, je Zugriff freigegeben)	eigene Programme
Betriebssystem-Schlüsselbund füllen	Windows Credential Manager, Gnome-Keyring

Was alles hineingehört (Eintragstypen)

Jeder Eintrag hat einen Typ; der Typ entscheidet, welche Wege oben ihn anbieten.

- Benutzername + Passwort, mit Adresse der Webseite und Verlauf alter Passwörter
- TOTP- und **HOTP**-Saat (Google Authenticator & Co., Import per QR)
- SSH-Schlüssel (im SE050 erzeugt oder eingelesen)
- Zertifikate mit Schlüssel: S/MIME, Code-Signatur, VPN, Windows-Anmeldung
- PGP-Schlüssel für E-Mail
- API-Schlüssel, Token, `.env`-Inhalte, Datenbank- und Server-Zugänge, kubeconfig
- Wiederherstellungscodes (die Zettel, die man bei 2FA bekommt)
- **Krypto-Seed-Phrasen** (BIP39, 12/24 Wörter)
- Softwarelizenzen und Seriennummern
- Bankkarten und Kreditkarten (für Formularausfüllen)
- Identitäten: Name, Adresse, Telefon, Geburtsdatum, Ausweis- und Versicherungsnummern
- WLAN-Zugänge, VPN-Konfigurationen
- Sichere Notizen und kleine Dateien
- Vorlagen für neue Einträge, damit nichts vergessen wird

Was man damit tun kann (über die Zugänge oben)

- Sich bei Webseiten anmelden: per Passkey, per Passwort, per TOTP
- **Fremde Passwortmanager entsperren und absichern**: 1Password, Bitwarden, KeePassXC, Proton Pass — der Stick ist deren zweiter Faktor bzw. ihr Schlüssel
- Am Rechner anmelden: Windows-Anmeldung, Linux-`sudo`/Login (PAM), Mac
- **Festplatte entsperren**: BitLocker, LUKS, VeraCrypt — mit dem Stick statt Kennwort
- **Krypto-Überweisungen signieren** (Bitcoin, Ethereum und weitere) — mit Anzeige von Empfänger und Betrag am Gerät
- Dateien verschlüsseln und entschlüsseln, mit Zeitstempel signieren
- E-Mails signieren und verschlüsseln, Git-Commits signieren, Code signieren
- SSH und Git ohne Schlüsseldatei auf der Platte
- Passwörter an jede Software weitergeben, auch an solche, die nichts davon weiß
- **Als Dongle eigene Software schützen**: läuft nur mit gestecktem Stick, und geschützte Teile sind ohne ihn gar nicht lesbar
- **Server absichern**: Knotenschlüssel im Stick statt auf der Platte, Aufträge signieren, Server-zu-Server verschlüsseln (Anwendungsfall NGC)

Bluetooth für FIDO lassen wir weg: Browser unterstützen es nicht verlässlich. BLE dient nur unserem eigenen Programm.

Was Nutzer sonst noch brauchen (sonst ist es unbenutzbar)

Stick verloren — das Wichtigste überhaupt. Ohne Vorsorge sind alle Konten weg.

- **Stick zu Stick, direkt und ohne Wolke** — der Hauptweg: Zwei Sticks werden gepaart (einer wird kurz Zentrale, der andere Peripherie über Bluetooth; über USB am Rechner geht es auch, dann sieht der Rechner nur Kryptotext). Beide zeigen dieselbe Zahl, beide bestätigen mit Tastendruck und Finger. Danach wandert der ganze Tresor verschlüsselt hinüber, Ende zu Ende zwischen den beiden Sicherheitschips.
- **Was gespiegelt wird**: Passwörter, TOTP/HOTP, Notizen, Dateien, SSH-Schlüssel, Zertifikate — und, wenn man es einschaltet, auch die **Passkeys**. Dafür leiten beide Sticks ihre FIDO-Schlüssel aus einem gemeinsamen Grundgeheimnis ab, das beim Paaren übertragen wird. Sonst ist der Zweitstick bei Verlust wertlos. Pro Profil umschaltbar: „gespiegelt" (bequem, Zweitstick ist Ersatz) oder „nur dieses Gerät" (sicherer, Schlüssel verlässt den Chip nie) — mit klarer Ansage, was das bedeutet. Fingerabdrücke werden **nie** übertragen.

- Das Programm führt eine Liste, bei welchem Dienst welcher Stick registriert ist, und erinnert, wenn ein Dienst nur einen kennt.
- **Verschlüsselte Sicherung** als Datei auf Platte oder NAS, Wiederherstellung nur mit Hauptkennwort — zusätzlich zum Stick-zu-Stick-Weg, für den Fall, dass beide Sticks weg sind. Dieselbe Datei kann einen neu gekauften Stick aufsetzen.
- **Sperren**: verlorenes Gerät im Programm als verloren markieren, Liste der dort registrierten Dienste ausgeben, damit man sie einzeln widerrufen kann
- **Notfall/Erbe**: Hauptkennwort in Teile zerlegt (Shamir), damit Angehörige mit mehreren Umschlägen herankommen

Alltag

- Passwortgenerator: Länge in **Zeichen** (nicht Ziffern) und Passphrase als Vorschlag
- Doppelte, zu kurze und zu alte Passwörter finden
- Ordner, Etiketten, Favoriten, Suche; Verknüpfung Eintrag ↔ Webseite
- **Phishing-Schutz**: die Erweiterung füllt nur auf der hinterlegten Adresse aus
- Mehrere Profile auf einem Stick (privat/Firma) mit getrennten PINs
- Teilen einzelner Einträge mit einem zweiten Stick (Familie), später
- Oberfläche in fünf Sprachen, große Schrift

Sicherheit im Betrieb

- Protokoll im Gerät: wann welcher Eintrag an welche Software ging
- Selbstlöschung nach n Fehlversuchen, abschaltbar
- Echtheitsprüfung des Geräts (Attestierung mit Gerätezertifikat), Siegel an der Verpackung
- Firmware-Update signiert und im Programm sichtbar

Freigabe-Regeln (wann gibt das Gerät etwas heraus)

- PIN einmal je Sitzung, Tastendruck je Vorgang
- Automatisches Sperren nach Zeit, sofortiges Sperren beim Abziehen
- Besonders geheime Einträge: Freigabe nur an gekoppelten Rechnern
- Schreibgeschützter Modus (Gast), in dem nichts geändert werden kann
- Das Programm zeigt immer, **welche** Software gerade fragt

Uhr Mit Akku läuft die Uhr weiter (RTC des nRF52840). Sie wird beim Anstecken nachgestellt; weicht sie zu weit ab, sagt die Oberfläche das. HOTP bleibt als Ausweichweg für Dienste ohne Zeitbezug.

Portable Software auf dem Stick Der Stick meldet sich zusätzlich als kleines, **schreibgeschütztes** Laufwerk mit `osk-win.exe`, `osk-linux`, `osk-mac` — einstecken und starten, keine Installation. Dafür passt der vorhandene Weg aus `Z:\GitHub\portable-starter`. Grenze, die ins Handbuch gehört: ohne Adminrechte gehen Kommandozeile, Oberfläche, ssh-agent, PKCS#11 und die Browser-Erweiterung. Windows-Minidriver, Festplattenentsperrung und PAM brauchen einmalig Adminrechte — das ist Betriebssystem-Sache, nicht unsere. Das Laufwerk ist abschaltbar und nur über ein signiertes Firmware-Update beschreibbar, damit es kein Einfallstor wird.

Drumherum

- Gehäuse in Fusion 360 (`Z:\GitHub\fusion360`), 3D-Druck, danach Spritzguss-fähig
- USB-Kennung über pid.codes (eigene VID ist unbezahlbar), AAGUID eintragen
- Für Verkauf später: CE/FCC/RoHS, FIDO-Zertifizierung kostet Geld — läuft erst, wenn das Gerät steht
- Lizenz, Spenden-Link, Bild und Facebook-Text zum neuen Repo

Alle Anmeldeverfahren, die das Gerät können muss

Was schon geplant war: FIDO2/WebAuthn/Passkeys, PIV, OpenPGP, TOTP, HOTP. Was noch fehlte und mit hineingehört:

- **U2F / CTAP1** — alte Dienste, die noch kein FIDO2 sprechen

- **Yubico-OTP** (44 Zeichen, als Tastatur getippt) — viele Firmendienste und Passwortmanager erwarten genau das
- **Challenge-Response HMAC-SHA1** — damit entsperren KeePassXC und VeraCrypt
- **FIDO2 hmac-secret** — damit entsperren BitLocker, LUKS, `systemd-cryptenroll`
- **TLS-Client-Zertifikate (mTLS)** — Webseiten, VPN, Firmennetze
- **EAP-TLS** — WLAN-Anmeldung mit Zertifikat statt Kennwort
- **Kerberos/PKINIT über PIV** — Smartcard-Anmeldung in Windows-Domänen, Windows Hello for Business
- **SSH mit Zertifikaten** (von einer eigenen CA signiert), zusätzlich zu SSH-Schlüsseln und FIDO-SSH
- **PAM** für Linux-Anmeldung und `sudo`, **macOS**-Anmeldung und FileVault
- **RFC-3161-Zeitstempel** bei Signaturen (geht jetzt, weil die Uhr läuft)
- **NDEF-Antippen**: Stick ans Handy halten → öffnet eine Adresse oder tippt ein Einmalkennwort, ohne jede App
- **Eigene Zertifizierungsstelle im Gerät**: eigene CA erzeugen und damit Zertifikate für eigene Geräte und Dienste ausstellen

Dongle: Software schützen und Server absichern

Software läuft nur mit Stick (Lizenz-Dongle)

- Das Programm fragt den Stick regelmäßig: Zufallswert hin, Signatur zurück, geprüft gegen das Gerätezertifikat. Jede Frage ist neu, also lässt sich die Antwort nicht aufzeichnen und wiedergeben. Beim Abziehen bricht das Programm ab.
- **Ehrliche Grenze**: eine reine Ja/Nein-Abfrage lässt sich aus dem Programm herausschneiden. Echter Schutz ist nur die zweite Stufe: ein Schlüssel, der den Stick nie verlässt, und ohne den Teile des Programms oder seiner Daten gar nicht erst lesbar werden. Dafür entschlüsselt der Stick auf Anfrage — und nur bei gesteckter Hardware — die geschützten Teile.
- Dazu eine kleine Einbindungs-Bibliothek `osk-guard` für C, Go, PHP, .NET und Python, damit fremde Programme das in wenigen Zeilen nutzen können.
- An die Programmdatei gebunden: ihre Prüfsumme geht in die Frage ein, eine manipulierte Datei bekommt keine gültige Antwort.
- Lizenzen im Tresor: Laufzeit, Ausbaustufen, Anzahl Plätze — vom Hersteller signiert, vom Programm prüfbar.

Server zu Server (Anwendungsfall NGC)

- Der dauerhafte Knotenschlüssel liegt im SE050 statt in einer Datei auf der Platte. Wer den Server kopiert, kopiert den Schlüssel nicht mit.
- **Signieren**: Aufträge und Nachrichten werden mit Ed25519 im Stick unterschrieben, der Empfänger prüft gegen den öffentlichen Schlüssel.
- **Asymmetrisch verschlüsseln**: X25519-Schlüsselaustausch im Stick, daraus der Sitzungsschlüssel — der private Teil bleibt im Chip.
- Zugang für Server-Software über **PKCS#11** (OpenSSL, nginx, Java), über den **ssh-agent**-Weg oder über unsere lokale Schnittstelle — je nachdem, was die Software schon kann.
- Für Dauerbetrieb am Server: Modus „kein Tastendruck je Vorgang, einmal freigeschaltet" — ausdrücklich einzuschalten, im Protokoll vermerkt, und auf einzelne Schlüssel begrenzbar. Sonst stünde der Server bei jeder Signatur still.
- Mehrere Programme gleichzeitig: eine Warteschlange im Programm, damit sich die Zugriffe nicht ins Gehege kommen.

Krypto-Geldbörse (voll mit drin)

Das Gerät ist zugleich eine Hardware-Geldbörse — mit Display ist das sauber machbar, weil man vor dem Signieren Empfänger, Betrag und Gebühr am Gerät liest.

- **Seed**: BIP39 (12/24 Wörter) mit optionaler Passphrase, im Gerät erzeugt oder eingelesen; liegt im SE050 und verlässt es nie. Ableitung nach BIP32/BIP44 bzw. SLIP-0010 **im Gerät**.

- **Ketten**: Bitcoin (secp256k1, auch Litecoin u. ä.), Ethereum und alle EVM-Ketten (EIP-155, EIP-1559, EIP-712 für Nachrichten), Solana/Cardano über Ed25519. Weitere lassen sich nachrüsten, weil die Ableitung allgemein gebaut ist.
- **Anbindung an bestehende Geldbörsen-Programme**:
  - **PSBT** (BIP174) als Datei oder über unser Programm → Electrum, Sparrow, Bitcoin Core. Das ist der offene Standard und der Hauptweg.
  - Für MetaMask, Rabby und Electrum zusätzlich eine **Ledger-verträgliche APDU-Schicht** über USB-HID, damit diese Programme das Gerät ohne Umbau als Hardware-Geldbörse erkennen. Protokoll selbst geschrieben, kein Fremdcode.
  - Nachrichten signieren als Eigentumsnachweis
- **Blindsignieren wird verweigert**: was das Gerät nicht lesbar anzeigen kann, signiert es nicht ohne ausdrückliche Extrafreigabe.
- **Zu prüfen**: ob die gewählte SE050-Variante secp256k1 und Ed25519 beherrscht — sonst muss die Kurvenrechnung in der Firmware laufen, was den Schutz schwächt. Das entscheidet die Variantenwahl in Stufe 2.

CTAP-Erweiterungen, damit fremde Passwortmanager den Stick annehmen Damit man sich mit dem Stick bei **1Password, Bitwarden, KeePassXC, Proton Pass, Dashlane** anmelden *und* deren Tresor damit entsperren kann, braucht die Firmware:

- **hmac-secret** und **PRF** — damit entsperrt Bitwarden seinen Tresor über den Schlüssel; dasselbe Verfahren nutzen auch BitLocker und LUKS
- **largeBlob** — manche Dienste legen Daten im Schlüssel ab
- **credProtect**, **minPinLength**, Enterprise-Attestation — von Firmendiensten verlangt
- Dazu die bereits geplante Challenge-Response (HMAC-SHA1) für KeePassXC und VeraCrypt

Prüfung in Stufe 3 bzw. 5: echter Bitwarden-Tresor per Stick entsperrt, 1Password-Anmeldung mit Passkey vom Stick, KeePassXC-Datei geöffnet.

Fingerabdruck (entschieden: ja, von Anfang an): Sensor am SPI, Vergleich läuft **im Gerät**, die Abdruckdaten verlassen es nie und liegen verschlüsselt im Flash. Damit drei Faktoren: Besitz (Stick) + Wissen (PIN) + Körper (Finger). Im Alltag ersetzt der Finger die PIN-Eingabe am Rechner — ein Tastaturmitschnitt greift dann nicht mehr. Die PIN bleibt als Ausweichweg, wenn der Finger nicht erkannt wird (nasse Hände, Verletzung). FIDO2 kennt das als „interne Benutzerprüfung" und gibt es so an Browser und Betriebssystem weiter.

Maximale Sicherheit gegen Angreifer

Warum überhaupt ein zweiter Chip: Der nRF52840 allein ist angreifbar — seine Auslesesperre wurde schon durch Spannungs-Störimpulse überwunden. Darum liegt **kein** Geheimnis dauerhaft im nRF52840. Alles Geheime steckt im SE050, und der ist gegen genau solche Angriffe geprüft (EAL6+).

Gerät in fremder Hand

- Schlüssel entstehen im SE050 und verlassen ihn nie; der Tresor im Flash ist verschlüsselt, der Schlüssel dazu kommt je Sitzung aus dem SE050
- Die Leitung zwischen nRF52840 und SE050 wird verschlüsselt (SCP03) — wer die Platine anbohrt und mitliest, sieht nichts Brauchbares
- Fehlversuchszähler im SE050 (nicht im Flash, der sich zurücksetzen ließe), wachsende Wartezeit, optional Selbstlöschung
- Auslesesperre und Debug-Schnittstelle dauerhaft zu, Bootloader nur signiert
- Kritische Entscheidungen doppelt geprüft, Vergleiche mit gleicher Laufzeit, Zufallsverzögerung — gegen Störimpuls- und Zeitmessangriffe
- RAM wird nach jedem Vorgang gelöscht, kein dynamischer Speicher in Krypto-Pfaden, Stapelschutz und MPU an

Verseuchter Rechner

- Jede Herausgabe braucht einen Tastendruck am Gerät — Schadsoftware allein bekommt nichts
- Das Programm zeigt, welche Software fragt; besonders geheime Einträge nur an gekoppelten Rechnern
- PIN geht über den verschlüsselten FIDO-PIN-Kanal, nicht als Klartext-Tastatur
- Das Stick-Laufwerk ist schreibgeschützt — es kann kein BadUSB-Einfallstor werden
- Sperren bei Inaktivität und beim Abziehen

Netz und Funk

- Passkeys sind an die Domäne gebunden, die Erweiterung füllt nur dort aus — Phishing greift nicht
- Bluetooth nur mit Zahlenvergleich und Verschlüsselungspflicht, kein „Just Works", Kopplung nur nach Tastendruck, wechselnde Funkadresse gegen Verfolgung
- Auch bei NFC braucht jeder Vorgang den Tastendruck — gegen Weiterleitungsangriffe aus der Tasche
- Kein Server, keine Wolke, keine Telemetrie: es gibt nichts zentral zu erbeuten

Unser eigener Code

- Die Parser für APDU, CTAP und NDEF werden gefuzzt (das ist der Teil, den ein Angreifer zuerst erreicht), dazu statische Prüfung und die FIDO-Testwerkzeuge
- Bedrohungsmodell öffentlich in `docs/`, Quelltext offen, reproduzierbarer Bau mit Prüfsummen — Sicherheit nicht durch Verstecken
- Lieferkette: Gerätezertifikat, Siegel, Prüfung beim ersten Start

Aktualisierung

Firmware im Stick

- MCUboot mit zwei Steckplätzen (A/B): Neues wird daneben geschrieben, geprüft, dann umgeschaltet. Bricht es ab, startet die alte Fassung weiter — der Stick kann nicht totgeschrieben werden.
- Nur **signierte** Pakete (ed25519). Der Signaturschlüssel liegt offline, nicht im Repo. Rückschritt auf ältere Fassungen ist gesperrt (Zähler im SE050).
- Update über USB aus dem Programm, später auch über Bluetooth. Der Stick fragt vorher mit Tastendruck.
- Der Tresor bleibt erhalten: das Datenformat hat eine Version, und beim ersten Start nach dem Update wandert es mit.
- Notfall: Bootloader-Betrieb per Tastendruck beim Einstecken, falls doch etwas klemmt.

Programme am Rechner

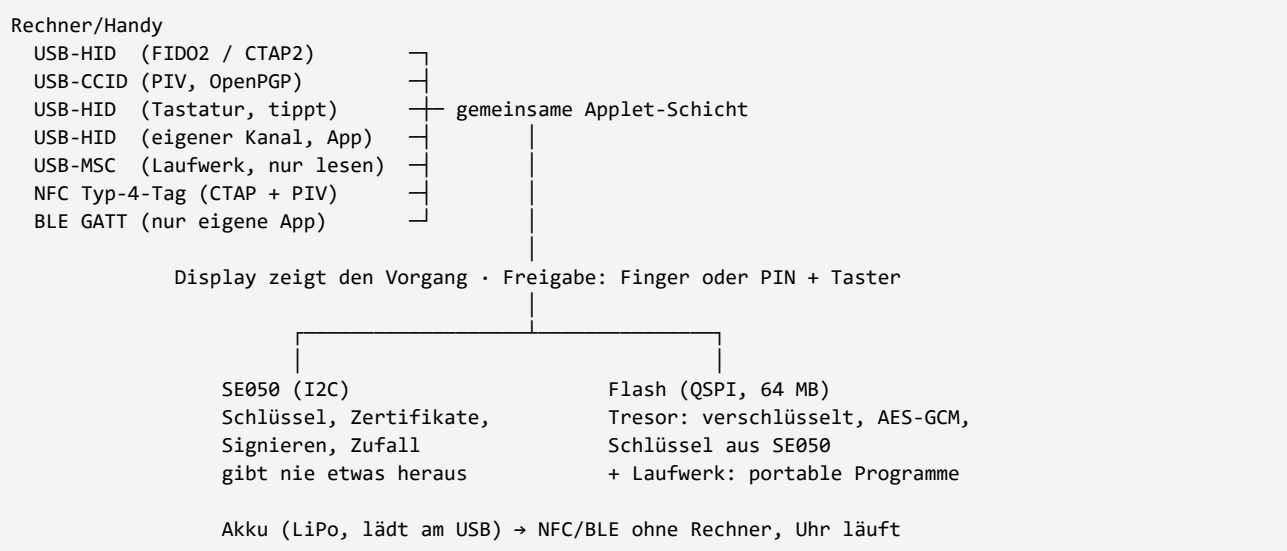
- Sie aktualisieren sich selbst, wie `ulh`/`uwh`: Fassung bei GitHub nachsehen, signiertes Paket holen, sich ersetzen. Abschaltbar, und es geht auch per Datei ohne Internet.
- Nach dem Update schreibt das Programm die neuen portablen Fassungen auch auf das Stick-Laufwerk, damit der Stick nie veraltete Software mitträgt.
- Browser-Erweiterung läuft über die Stores (Chrome, Firefox, Edge) mit deren eigenen Freigabezeiten; Handy-Apps über App Store und Play Store.
- Altes Gerät mit neuem Programm und umgekehrt: die Protokollversion wird beim Verbinden abgesprochen, und die Oberfläche sagt klar, was fehlt.
- Das SE050 selbst wird nicht aktualisiert — es ist ein fester Chip.

Was noch dazugehört

- **Erster Start**: Einrichtungsassistent — PIN setzen, Zweitgerät anlegen, Sicherung speichern, Notfallzettel ausdrucken. Nicht überspringbar.
- **Zurücksetzen** auf Werkszustand: alles löschen, nur mit langem Tastendruck und getippter Bestätigung
- **Keine Telemetrie**, keine Konten, kein Server. Das Gerät funktioniert ohne uns.
- **Reproduzierbarer Bau**: gleiche Quelle ergibt gleiche Datei, Prüfsummen bei jeder Veröffentlichung — damit jeder nachrechnen kann, dass die Firmware aus diesem Quelltext stammt
- `SECURITY.md` mit Meldeweg, und ein Platz für Sicherheitsmeldungen an Nutzer

- Handbuch im Wiki, fünf Sprachen, mit Bildern
- Mehrere Sticks eines Nutzers abgleichen (über eine Datei, nicht über eine Wolke)

Aufbau



Repo

Neu: `Z:\GitHub\opensecurekey`, öffentlich, streng nach `claude-md\repo.md`, README englisch, Autor „Sorglos Thomas Weirich", vierstellige Version je Teilprogramm (Start `0.0.0.0`), Oberflächen in fünf Sprachen.

```
opensecurekey/
├── README.md · CHANGELOG.md · .gitignore
├── apps/
│   ├── firmware/        nRF Connect SDK (Zephyr), VERSION, src/, include/, test/
│   │   ├── src/classes/ se050/ · ctap/ · applets/ · transport/ · store/ · wallet/
│   │   └── src/views/   Display-Bilder
│   ├── desktop/         Verwaltungsprogramm (Go) + lokale Weboberfläche, `osk`
│   ├── web/             Browser-Erweiterung
│   └── mobile-ios/ · mobile-android/   (später)
├── shared/              Protokoll Firmware ⇔ Programm, Tresor-Datenformat
├── hardware/            Schaltplan, Layout, Gerber, Stückliste
├── cad/                 Gehäuse (Fusion 360)
├── docs/                Architektur, Bedrohungsmodell, Messprotokolle, Handbuch
└── tools/               build.py, set_version.py, flash.py, release.py
```

Fremdcode wird nicht als Abhängigkeit eingebunden: SE050-Treiber, Applets und Protokolle werden selbst geschrieben; nur das Hersteller-SDK (nRF Connect SDK) bleibt Werkzeug, wie beim oltd die ESP-IDF.

Arbeitsaufteilung (`#ntw`)

Diese Sitzung bleibt Master: verteilt, sammelt, führt zusammen, fragt als Einzige. Jeder Strang bekommt einen eigenen Worktree `D:\claude\opensecurekey-<name>` und einen eigenen Zweig, fasst **nur seine eigenen Ordner** an und hält die Pflichten aus `#nt` ein (von selbst melden, committen, zusammenführen, Worktree entfernen, Sitzung archivieren; bei Konflikt offen bleiben und melden). Jede Übergabe nennt `Z:\GitHub\claude-md\*.md` als verbindlich.

Welle 0 — Master allein, muss zuerst fertig sein Repo anlegen, Struktur nach `repo.md`, README/CHANGELOG/.gitignore, `VERSION` je Teilprogramm, und — das Entscheidende — die **inneren Schnittstellen als Header** (`se050.h`, `ctap.h`, `store.h`, `transport.h`, `shared/protokoll.md`). Erst dadurch werden die Stränge voneinander unabhängig. Committen und pushen.

Welle 1 — sechs Stränge gleichzeitig, alle ohne Hardware arbeitsfähig

Strang	Ordner	Inhalt
`se050`	`src/classes/se050/`	T=1' über I2C, APDU nach AN12413, SCP03, PC-Tests gegen Testdaten
`ctap`	`src/classes/ctap/`	CTAP2-Kern, hmac-secret/PRF/largeBlob/credProtect, USB-HID-Transport
`tresor`	`src/classes/store/`, `shared/`	Eintragstypen, AES-GCM, Formatversion und Wanderung, Sicherungsdatei, Shamir
`desktop`	`apps/desktop/`	Go-Gerüst, HID-Schicht für Windows/Linux/Mac, lokale Weboberfläche, fünf Sprachen
`hardware`	`hardware/`, `docs/`	Schaltplanentwurf, Teileauswahl, Energiebudget, Stückliste, Fingerabdrucksensor suchen
`werkzeuge`	`tools/`, `.github/`	build.py, set_version.py, flash.py, release.py, CI nach `CI.md`

Welle 2 — erst wenn Welle 1 zusammengeführt ist `smartcard` (PIV, OpenPGP, CCID, Yubico-OTP, Challenge-Response) · `krypto` (BIP32/39/44, PSBT, Ethereum, Ledger-APDU-Schicht) · `funk` (NFC-Typ-4-Tag, BLE-GATT, Kopplung) · `display` · `anbindungen` (PKCS#11, ssh-agent, Browser-Erweiterung, Minidriver) · `dongle` (Challenge-Response, `osk-guard` in fünf Sprachen, Server-Dauerbetrieb).

Welle 3 — nur mit Hardware auf dem Tisch Alle Messungen aus dem Prüfabschnitt, Stufe für Stufe.

Regeln für alle Stränge: ein Zweig je Strang, nie zwei Stränge in denselben Dateien, `git add <Pfade>` und `git commit -- <Pfade>` (der Index ist gemeinsam), und was nicht am Gerät lief, wird auch nicht als geprüft gemeldet.

Stufen

Jede Stufe endet mit einer Messung **am Gerät**, nicht mit einem grünen Bau.

1 — Teststand (Werkzeug steht, Hardware antwortet) nRF52840-DK (PCA10056) + SE050-Breakout (OM-SE050ARD passt auf den Arduino-Header). nRF Connect SDK einrichten, Bau per `tools/build.py` aus PowerShell (idf-Lektion aus oltd: Aufruf über cmd). Prüfung: USB wird erkannt, Taster/LED laufen.

2 — SE050 lebt Eigener I2C-T=1'-Treiber, dann APDU-Schicht, dann der verschlüsselte Kanal (SCP03). Prüfung: Version lesen, ECC-P256-Schlüssel im Chip erzeugen, Signatur erzeugen, Signatur am PC gegen den öffentlichen Schlüssel prüfen; Mitlesen am I2C zeigt keinen Klartext. Erst wenn das steht, hat alles weitere einen Sinn.

3 — FIDO2 über USB (die Stufe, nach der das Gerät schon nützt) CTAP2-Kern + USB-HID-Transport, Benutzerbestätigung über den Taster, PIN mit Fehlversuchszähler. Prüfung: echter Passkey bei einer Webseite anlegen und sich anmelden, in Chrome und mit Windows Hello.

4 — NFC Typ-4-Tag-Transport (ISO-DEP), dieselbe CTAP-Schicht. Prüfung: Anmeldung am Handy, Gerät am USB-Netzteil versorgt.

5 — Smartcard USB-CCID + PIV-Applet, dann OpenPGP, dazu U2F/CTAP1, Yubico-OTP und Challenge-Response. Prüfung: Zertifikat erzeugen, E-Mail in Betterbird signieren, SSH-Anmeldung, Datei signieren, KeePassXC entsperren — ohne eigenen Treiber.

6 — Tresor: alles speichern Externer Flash, verschlüsselter Speicher mit typisierten Einträgen (Passwort, Benutzername, TOTP, SSH-Schlüssel, Zertifikat, API-Schlüssel, Notiz, kleine Datei), TOTP-Rechnung, Tastatur-Ausgabe, eigener HID-Kanal. Prüfung: Passwort auf Tastendruck getippt, TOTP-Code gegen Google Authenticator verglichen, ein SSH-Schlüssel im SE050 erzeugt und ausgelesener öffentlicher Teil geprüft.

7 — Verwaltungsprogramm (Windows, dann Linux, dann Mac) Go, Oberfläche als lokale Webseite auf Loopback mit Origin-Prüfung, kein Token (Hausregel). Einträge jeder Art anlegen, ändern, suchen; Zertifikate erzeugen; Geräte koppeln; Firmware aufspielen; Sicherung speichern/laden; Import aus KeePass/CSV und vorhandene SSH-Schlüssel übernehmen.

8 — Anbindungen: an jede Software weitergeben Die Tabelle oben, Stück für Stück, jede mit eigener Messung: `osk`-Kommandozeile → ssh-agent → PKCS#11-Bibliothek → Browser-Erweiterung → Windows-Minidriver/CNG → lokale Schnittstelle → Mac CryptoTokenKit. Jeder Zugriff fragt nach Freigabe am Gerät (Tastendruck), und das

Programm zeigt, welche Software gerade fragt. Prüfung: `git push` über unseren Agenten, `openssl` signiert über PKCS#11, Firefox nutzt unser Zertifikat, Anmeldung im Browser über die Erweiterung ausgefüllt.

9 — Bluetooth und Kopplung Eigener GATT-Dienst, LESC mit Zahlenvergleich, Kopplung erst nach Tastendruck am Gerät, Kopplungen im Programm sichtbar und einzeln löschbar.

10 — Eigene Platine Schaltplan und Layout über die EasyEDA-Brücke: vorzertifiziertes nRF52840-Modul, SE050, QSPI-Flash 64 MB (Tresor **und** portable Programme), USB-C-Stecker, Laderegler + Akkuschutz + LiPo-Anschluss (nicht hinter der NFC-Antenne), OLED am SPI, Fingerabdrucksensor am SPI, Taster, LED, NFC-Antenne im Layout. DRC auf 0, Gerber, Stückliste. Pad-Maße aus dem Gerber prüfen, nicht aus der Bibliothek; Netzfahnen mit Stichleitung absetzen.

11 — Display OLED am SPI, Anzeige des laufenden Vorgangs (Dienst, anfragende Software, Betrag), Schriften für fünf Sprachen, Freigabe erst nach dem Lesen. Prüfung: bei einer Anmeldung steht die richtige Webseite auf dem Gerät, bei einer untergeschobenen Anfrage die falsche — und man sieht es.

12 — Fingerabdruck Sensor am SPI, Anlernen und Vergleich im Gerät, Abdruckdaten verschlüsselt im Flash. Als interne Benutzerprüfung in CTAP2 gemeldet, PIN bleibt Ausweichweg. Prüfung: Anmeldung im Browser allein mit Finger, PIN-Weg nach drei Fehlversuchen.

13 — Krypto-Geldbörse BIP39/BIP32-Ableitung im SE050, PSBT für Bitcoin, Ethereum-Transaktionen und EIP-712, Anzeige von Empfänger/Betrag/Gebühr auf dem Display, Ledger-verträgliche APDU-Schicht. Prüfung: Testnetz-Überweisung aus Electrum und aus MetaMask signiert und gesendet, Anzeige am Gerät stimmt mit der Überweisung überein.

14 — Dongle und Server Challenge-Response gegen das Gerätezertifikat, Entschlüsselung geschützter Programmteile, Bibliothek `osk-guard` (C, Go, PHP, .NET, Python), Lizenzeinträge, Dauerbetriebsmodus für Server mit Protokoll. Prüfung: ein Testprogramm startet nur mit Stick und bricht beim Abziehen ab; NGC signiert einen Auftrag über den Stick und ein zweiter Knoten prüft ihn; eine Datei wird zwischen zwei Servern mit X25519 aus dem Stick verschlüsselt.

15 — Stick als Laufwerk, Software portabel USB-Massenspeicher (schreibgeschützt) neben den anderen Schnittstellen, Programme für Windows/Linux/Mac als eine Datei je System darauf. Prüfung: an einem fremden Rechner ohne Adminrechte einstecken, starten, Passwort abrufen.

16 — iPhone, dann Android

17 — Alltag und Vorsorge Stick-zu-Stick-Übertragung (Prüfung: zweiter Stick, danach Anmeldung bei einem Dienst mit dem Zweitstick), Sicherungsdatei und Wiederherstellung auf einen dritten Stick, Dienste-Liste und Sperren, Generator, Doppelt-Prüfung, Ordner/Suche, Profile, Protokoll, Autosperre, Festplattenentsperrung, PAM-Anmeldung, Gehäuse aus Fusion 360.

18 — Aktualisierung und Herstellung MCUboot mit A/B und Signatur, Selbstaktualisierung der Programme, Wanderung des Tresor-Formats, Auslesesperre, Herstellungsablauf (Schlüssel und Gerätezertifikat beim ersten Start), reproduzierbarer Bau mit Prüfsummen.

Hinweis zur Reihenfolge: der **signierte Bootloader kommt früh** (schon mit Stufe 3 ins Abbild), sonst müsste später die Auslesesperre gelöst werden — und das löscht den Chip.

Sicherheit (Entwurfsregeln, gehören ins Bedrohungsmodell)

- Private Schlüssel entstehen **im** SE050 und verlassen ihn nie
- Flash-Inhalt ist mit AES-GCM verschlüsselt, Schlüssel wird je Sitzung aus dem SE050 geholt und danach aus dem RAM gelöscht
- PIN wird im SE050 gegengeprüft, Fehlversuchszähler dort; nach n Fehlversuchen sperrt der Chip
- Jede Herausgabe eines Geheimnisses braucht zwei Dinge: PIN (einmal je Sitzung) **und** Tastendruck am Gerät (je Vorgang)
- Firmware-Update nur signiert, Auslesesperre dauerhaft gesetzt
- Sicherung: verschlüsselte Datei, Wiederherstellung nur mit Hauptkennwort — bewusst **kein** Weg, der an den Schlüsseln im SE050 vorbeiführt

Prüfung

Nach jeder Stufe am echten Gerät, Messprotokoll in `docs/`:

- Stufe 2–3: `nrfutil`/Testprogramm, Signatur am PC nachgerechnet
- Stufe 3: Anmeldung bei einer echten Webseite, Chrome + Windows Hello
- Stufe 5: `pkcs11-tool`/Windows-Zertifikatsspeicher, Signatur in Betterbird
- Stufe 6: TOTP-Code gegen eine zweite App verglichen
- Stufe 7: Programm auf Windows und Linux gebaut und gegen dasselbe Gerät betrieben
- Stufe 8: je Anbindung ein echter Vorgang (`git push`, `openssl`, Firefox, Browser- Erweiterung an einer echten Anmeldeseite)
- Stufe 10: DRC, Gerber-Nachmessung

Vier Stufen von „geprüft" gelten: geschrieben / übersetzt / im Abbild / am Gerät gelaufen — im Protokoll steht, welche.

Zusätzlich dauerhaft mitlaufend: Fuzzing der APDU-, CTAP- und NDEF-Parser, statische Prüfung, und ein Angriffs-Prüfbogen in `docs/` (Fehlversuche, Mitlesen am I2C, Abziehen mitten im Vorgang, Rückschritt-Firmware, Phishing-Adresse).

Offene Punkte (Prüfung des Gesamtkonzepts)

Widersprüche, die vor dem Schaltplan geklärt sein müssen

1. **Flash: entschieden, 64 MB statt 16 MB.** Drei portable Programme (Windows, Linux, Mac) sind je rund 5–15 MB; mit 16 MB passt der Tresor nicht daneben. Programme bleiben klein (Go ohne cgo, gestrippt).
2. **Display: entschieden, kleines OLED kommt rein.** Damit zeigt das Gerät selbst, was es freigibt — Webseite, anfragende Software, bei Überweisungen der Betrag. Erst damit muss man dem Rechner nicht glauben. Folgen: SPI-Anschluss im Schaltplan, Gehäusefenster, eigene Firmware-Stufe mit Schriftarten in fünf Sprachen, und etwas mehr Stromverbrauch im Energiebudget.
3. **Akku dämpft NFC.** Die Zelle darf nicht hinter der Antenne liegen. Betrifft Platinenumriss und Gehäuse — muss im Layout von Anfang an berücksichtigt werden.
4. **„Gespiegelte Passkeys" und Zertifizierung passen nicht zusammen.** Wer Passkeys auf zwei Geräte spiegelt, darf keine gerätegebundene Bescheinigung behaupten. Für dich privat kein Problem; eine FIDO-Zertifizierung später wäre damit aber verbaut. Zu entscheiden, sobald Verkauf im Raum steht.

Technische Punkte, die ich noch nicht beantworten kann

5. **Fingerabdrucksensor**: Gute Module rechnen den Vergleich selbst; die sind in Kleinmengen schwer zu bekommen und oft nur mit Geheimhaltungsvertrag. Den Vergleich selbst zu schreiben ist ein eigenes Projekt. → Vor Stufe 11 muss ein beschaffbarer Sensor mit offener Beschreibung gefunden sein, sonst verschiebt sich das. Platz und Anschluss sind trotzdem von Anfang an im Schaltplan.
6. **Fünf USB-Schnittstellen gleichzeitig** (FIDO-HID, Tastatur, eigener Kanal, Smartcard, Laufwerk): das muss früh an echten Rechnern geprüft werden, manche Systeme und BIOSe stolpern darüber. Gegenmittel: Schnittstellen einzeln abschaltbar machen (Profile im Programm). Prüfung gehört in Stufe 3.
7. **Funkzulassung: entschieden, vorzertifiziertes Modul** (Raytac/Fanstel u. ä.) statt nacktem nRF52840 — sonst müsste die Platine selbst als Funkgerät geprüft werden, und das ist teuer. Kostet ein paar Millimeter.
8. **SE050: geprüft, passt** (Datenblatt Rev. 3.8, Tabelle 2): Koblitz/secp256k1 für Bitcoin und Ethereum, Ed25519 und Curve25519, NIST bis 521 Bit, Brainpool, RSA bis 4096, HKDF/PBKDF2/HMAC/CMAC, SHA-1 bis SHA-512, echter Zufallsgenerator, SCP03, 50 kB Nutzspeicher, 3 × 3 mm. Die 50 kB sind der Grund, FIDO-Schlüssel **abzuleiten** statt einzeln zu speichern — so sind beliebig viele Dienste möglich. Offen bleibt nur: welche Variante genau. `OM-SE050ARD` (SE050C) ist abgekündigt, lieferbar ist `OM-SE050ARD-E` (SE050E, neuere Generation). Ob die E-Variante alle Kurven mitbringt, fragt Stufe 2 als Erstes direkt am Chip ab.
9. **Energiebudget**: Akkugröße, Bluetooth-Bereitschaft, Stromstoß beim Fingerabdruck-Scan, Ladestrom am USB — muss gerechnet werden, bevor die Zelle gewählt wird. Wenn der Akku leer ist, läuft alles weiter am USB, nur die Uhr steht und muss nachgestellt werden.

10. **Laufzeit und Alterung**: LiPo in einem Schlüsselanhänger hält wenige Jahre. Tauschbar bauen oder als Verbrauchsteil ansagen.

Grenzen, die ins Bedrohungsmodell gehören (nicht lösbar, nur benennen)

11. Das SE050 schützt gegen das Auslesen von Chips — **nicht** gegen eine untergeschobene eigene Firmware. Dagegen hilft nur der signierte Bootloader.
12. Die PIN wird am Rechner eingegeben; ein Tastaturmitschnitt sieht sie. Darum ist der Fingerabdruck der eigentliche Alltagsweg.
13. Mehrere Profile auf einem Gerät sind in FIDO nicht vorgesehen — beim Profilwechsel muss man sich bei den Diensten neu anmelden.
14. Hauptkennwort vergessen und beide Sticks verloren = alles verloren. Deshalb ist der Notfallzettel im Einrichtungsassistenten nicht überspringbar.

Organisatorisch offen

15. Lizenz (Software und Hardware getrennt), Repo-Name und Markenname
16. USB-Kennung über pid.codes beantragen
17. Größenordnung: das sind mehrere Monate Arbeit. Nach **Stufe 3** hast du aber schon einen brauchbaren FIDO2-Schlüssel, nach **Stufe 6** einen Passwortspeicher. Alles danach ist Ausbau.

Erster Schritt

Stufe 1 braucht Hardware auf dem Tisch.

GeeekPi nRF52840 Micro Dev Kit USB Dongle (Nachbau des MakerDiary MDK) — brauchbar, aber nur als Zweitgerät:

- nRF52840 mit 1 MB Flash, echter USB-Stecker, UF2-Bootloader (Firmware per Ziehen und Ablegen aufspielen, kein Zubehör nötig), Taster und RGB-LED an Bord, 42 × 18 mm — die Bauform, die wir später wollen
- I2C vorhanden → SE050 lässt sich anschließen; Stufen 1–3 (SE050, FIDO2 über USB) gehen damit
- **kein Debugger an Bord**, SWD nur als Kantenpads. Bei Sicherheitsfirmware mit Auslesesperre ist Fehlersuche ohne Debugger mühsam
- **nur 12 GPIOs** — für Display *und* Fingerabdruck *und* NFC später zu knapp
- ob die NFC-Pins (P0.09/P0.10) herausgeführt sind, sagt die Beschreibung nicht; eine NFC-Antenne lässt sich also vielleicht gar nicht anschließen

Darum als Haupttestst and das nRF52840-DK: Debugger an Bord, fester NFC-Antennenanschluss, alle Anschlüsse frei, und der OM-SE050ARD steckt direkt oben drauf. Der Dongle bleibt daneben nützlich — für Stick-zu-Stick-Übertragung braucht es später sowieso zwei Geräte, und er zeigt früh, ob alles in die Zielbauform passt.

Bestellliste — jetzt (Teststand, Stufen 1–5)

#	Teil	Bestellnummer	Woher	ca.
1	Haupttestst and nRF52840-DK	`nRF52840-DK` / PCA10056	Mouser, DigiKey, Reichelt	50 €
2	SE050-Platine am Arduino-Header	`OM-SE050ARD-E`	DigiKey, Future	70–85 €
2b	*Günstigere Alternative*: SE050C2 auf Click-Platine	MikroElektronika „Secure 4 Click"	MikroE, Reichelt	30 €
3	Zweitgerät in Zielbauform	GeeekPi nRF52840 MDK USB Dongle	Amazon	25 €
4	NFC-Antenne 13,56 MHz für das DK (liegt **nicht** bei)	Flex-Antenne mit 2-pol. Anschluss	Mouser, Amazon	8 €
5	OLED 0,96", SSD1306, **SPI**-Ausführung (7-polig)	—	Amazon, Reichelt	6 €

#	Teil	Bestellnummer	Woher	ca.
6	Logikanalysator 8 Kanal (um am I2C mitzulesen und SCP03 zu beweisen)	—	Amazon	15 €
7	Steckbrett, Dupont-Kabel, USB-Kabel (Micro-USB fürs DK, USB-C)	—	—	15 €

Zusammen rund **190–220 €** (mit Alternative 2b rund 150 €). Preise ungeprüft, vor dem Kauf kurz nachsehen — der SE050 ist das Teil mit dem größten Ausschlag, und `OM-SE050ARD` (ohne -E) ist abgekündigt, also **nicht** bestellen.

Noch nicht bestellen: Fingerabdrucksensor. Erst muss ein Modul gefunden sein, das den Vergleich selbst rechnet, in Kleinmengen zu bekommen ist und eine offene Beschreibung hat — das ist offener Punkt 5. Bis dahin genügt der Taster.

Bestellliste — später (eigene Platine, ab Stufe 10)

Erst nach dem Schaltplan bestellen, dann gegen die Stückliste:

- Funkmodul `MDBT50Q-1MV2` (Raytac, nRF52840, bereits funkzertifiziert)
- `SE050C2HQ1` oder die geprüfte E-Variante, QFN20 3 × 3 mm
- QSPI-Flash 64 MB, z. B. `W25Q512JVEIQ`
- Laderegler `MCP73831` + Akkuschutz (`DW01A` + `FS8205`), LiPo 100 mAh
- OLED-Modul als Einbauteil, Fingerabdrucksensor, Taster, RGB-LED
- USB-C-Stecker direkt an der Platinenkante, Abstimmkondensatoren für die NFC-Antenne (die als Leiterbahn im Layout entsteht)

Bis die Hardware da ist, läuft Welle 0 und dann Welle 1 — Repo, Schnittstellen, SE050-APDU-Schicht, CTAP-Kern, Tresorformat, Go-Programm, Schaltplanentwurf und Werkzeuge lassen sich alle ohne Gerät schreiben und am PC übersetzen. „Übersetzt" ist dabei nicht „gelaufen": alles Gemessene kommt erst in Welle 3.

Marktabgleich: was Yubico und die offenen Schlüssel können und wir nicht

Context

Der Entwurf oben ist aus eigenen Anforderungen gewachsen, nie gegen die Produktpalette gehalten, die er ersetzen soll. Thomas hat den Abgleich gegen **alle** Yubico-Produkte verlangt, einschließlich YubiHSM 2 — und das ist keine Schlüssel-Variante, sondern ein Server-Modul mit einem ganz anderen Sicherheitsmodell. Dazu die offenen Entwürfe Nitrokey, SoloKey und OnlyKey, die unserem Weg näher stehen als Yubico.

Ergebnis vorweg, und es hat zwei Hälften:

Bei den **Zusagen** ist der Entwurf oben breiter als jedes Einzelprodukt — Passwortspeicher, Geldbörse, Dongle und Anzeige hat kein Yubico-Produkt. Bei der **Umsetzung** fehlen dagegen Dinge, die bei Yubico Alltag sind, und drei davon sind keine Kleinigkeiten: **RSA ist ausdrücklich ausgeschlossen**, die CTAP-Befehle **CredentialManagement und Config sind nicht umgesetzt**, und der **Yubico-HID-Transport fehlt, weshalb KeePassXC und VeraCrypt mit diesem Bau nicht funktionieren** — obwohl die Challenge-Response darunter fertig ist.

Darum trennt jede Tabelle unten **Zusage** von **Stand**. Ein Vergleich, der geplante Fähigkeiten gegen ausgelieferte Produkte stellt, wäre eine Selbsttäuschung — und genau der Fehler, den die Hausregel „vier Stufen von geprüft" verhindern soll. Nichts in diesem Entwurf ist je auf Hardware gelaufen.

Die wichtigste Lücke gegenüber den *Produkten* stammt übrigens nicht von Yubico, sondern von OnlyKey (Abschnitt 1).

Die größten Lücken überhaupt stehen aber in Abschnitt 8, und sie kamen erst heraus, als Thomas nach Verschlüsselung, Signatur, Zertifikaten und Passwortspeicherung fragte: **drei Fähigkeiten, die der Entwurf oben ausdrücklich zusagt, existieren nirgends** — kein Weg, eine Datei zu verschlüsseln, keine Zertifikatserzeugung (nicht einmal ein CSR), kein einziger Importweg für vorhandene Passwörter. Nicht als Lücke benannt, nicht als offener Punkt. Gar nicht. Wer nur gegen Yubico vergleicht, findet das nicht, weil Yubico es auch nicht hat.

Dieser Abschnitt ist ein Dokumentationsauftrag, kein Umbau. Er endet mit einer neuen Datei und Ergänzungen an bestehenden — nicht mit Firmware-Arbeit. Was davon gebaut wird, entscheidet Thomas Punkt für Punkt danach.

1. Die eine echte Lücke: PIN-Eingabe am Gerät

Das Bedrohungsmodell oben benennt selbst eine Grenze:

> „Die PIN wird am Rechner eingegeben; ein Tastaturmitschnitt sieht sie. Darum ist der Fingerabdruck der eigentliche Alltagsweg."

OnlyKey löst das anders: die PIN wird auf dem Gerät selbst eingegeben, auf dessen eigener Tastatur, unerreichbar für einen verseuchten Rechner. Yubico hat das nicht, Nitrokey nicht, SoloKey nicht.

Und wir hätten die Hardware dafür schon: Anzeige plus Taster. Hardware- Geldbörsen geben PINs so ein — Ziffer am Display durchblättern, mit dem Taster bestätigen. Umständlicher als eine Tastatur, aber es nimmt dem Rechner das Geheimnis vollständig weg. Das ist der Punkt, an dem unser Entwurf durch *vorhandene* Teile sicherer wird als jedes Yubico-Produkt.

Zu klären ist nur die Bedienbarkeit: eine sechsstellige PIN mit einem Taster durchzublättern ist zäh. Zwei Taster oder ein Taster mit langem und kurzem Druck wären zu prüfen — eine Entwurfsentscheidung, keine Nebensache.

2. Was bei den Schlüssel-Produkten fehlt

2a. Die vier harten Lücken — hier fehlt Funktion, nicht Politur

Fähigkeit	Wo sie herkommt	Unser Stand, mit Belegstelle
RSA — PIV und OpenPGP mit RSA 2048/3072/4096	YubiKey 5.7 (nach DoD-Vorgabe von 2023)	**ausdrücklich ausgeschlossen.** „Elliptische Kurven nur… RSA-Schlüssel werden mit ‚function not supported' beantwortet statt vorgetäuscht" (`docs/smartcard.md`, `docs/se050.md` „Deliberate limits"). Damit scheitert jeder PIV- oder OpenPGP-Einsatz, der RSA verlangt — und das sind in Firmenumgebungen die meisten
`authenticatorCredentialManagement` **(0x0A)** — Passkeys am Gerät auflisten, zählen, einzeln löschen	YubiKey 5, Alltagsfunktion über Yubico Authenticator	**nicht umgesetzt** (`docs/ctap.md`). Ohne ihn kann niemand sehen oder aufräumen, was auf dem Gerät liegt
`authenticatorConfig` **(0x0D)** — Force PIN Change, Always Require UV, Enterprise Attestation einschalten	YubiKey Enhanced PIN / 5.7 Enterprise	**nicht umgesetzt** (`docs/ctap.md`). Damit fehlt auch **Enterprise Attestation ganz**: nirgends im Repo zugesagt, und Firmendienste, die ein echtes Attestierungszertifikat verlangen, bekommen keines
Yubico-HID-Report-Transport	YubiKey OTP-Applet	**nicht nachgebildet, und darum funktionieren KeePassXC und VeraCrypt mit diesem Bau nicht** (`docs/smartcard.md` „What is missing"). Bitter, weil die Challenge-Response HMAC-SHA1 darunter fertig ist — es fehlt nur der Weg dorthin. Grund: keine sauber befolgbare Beschreibung, und Nachbauen aus fremdem Quelltext verbietet die Hausregel

2b. Weitere Unterschiede

Fähigkeit	Wo sie herkommt	Unser Stand
Secure Static Password — festes Passwort auf Knopfdruck, ohne jede Software	YubiKey 5 (OTP-Steckplatz)	**nicht zugesagt.** Das Nächstliegende, „Gerät tippt als USB-Tastatur", ist selbst noch nicht gebaut
Echter Signaturzähler je Credential	YubiKey	**im RAM, startet bei jedem Einschalten bei null** (`docs/ctap.md`). Dienste, die den Zähler prüfen, sehen einen Rückschritt. Braucht eine Heimat im Flash
Rückdrehschutz des Speichers	YubiHSM 2 und YubiKey über Chipzähler	**offen.** Der Klartextkopf des Tresors ist nur durch Prüfsumme geschützt, nicht durch MAC — wer den Flash direkt schreibt, setzt einen älteren echten Zustand zurück, und nichts merkt es. Behebung braucht einen monotonen Zähler im Chip (`shared/vault-format.md` „Known limits")
Eindeutige Seriennummer je Gerät	YubiKey (über FIDO-HID und CCID lesbar)	**fehlt, mit sichtbarer Folge: GnuPG hält zwei unserer Sticks an einem Rechner für eine Karte**, weil die OpenPGP-AID überall dieselbe Nummer trägt. Beim Massenspeicher ist die `0001` dagegen Absicht (Spurvermeidung)
SCP03/SCP11 zum Schlüssel selbst — gesicherter Kanal zur Karte	YubiKey 5.7	**fehlt.** Wir nutzen SCP03 nur *innen*, zum Sicherheitschip. Nebenbefund aus der Recherche: Yubico liefert mit öffentlich bekannten SCP03-Standardschlüsseln aus, was den Kanal ohne Austausch wertlos macht — **genau dieselbe Falle hat unser Sicherheitschip**, und bei uns ist der Austausch beim ersten Start noch nicht gebaut (`docs/threat-model.md`)
YubiHSM Auth — Zugangsdaten für ein Server-HSM im Schlüssel	YubiKey ab 5.4.3	**fehlt.** Betrifft den NGC-Anwendungsfall unmittelbar
PIV-Attestierung je Steckplatz (F9 bescheinigt 9A/9C/9D/9E)	YubiKey ab 4.3	**fehlt.** Bei FIDO gibt es nur Selbst-Attestierung (`packed`, ohne `x5c`), die SE050-Objektattestierung wird geholt, **aber ihre Signatur nicht geprüft**
Festgelegte Kapazitäten — 100 Passkeys, 24 PIV-Zertifikate, 32 OATH, 190 Gesamtpool	YubiKey 5.7	**teils besser, teils unklar.** Zugesagt sind 768 Einträge je Profil und 3072 im Gerät — weit mehr als Yubico. Aber die Zahl der Passkeys ist in den eigenen Unterlagen **widersprüchlich** (siehe Abschnitt 6), und ein 32-Byte-Passwort belegt einen 8-KiB-Platz: 250-fache Verschwendung, als offener Punkt notiert
PIN-Regelwerk — Mindestlänge 6, Komplexität, nicht änderbar	YubiKey Enhanced PIN	**fehlt.** Unser Minimum sind 4 Codepunkte

Fähigkeit	Wo sie herkommt	Unser Stand
Verschlüsselter Massenspeicher bis 64 GB	Nitrokey Storage 2	**fehlt.** Unser Laufwerk ist schreibgeschützt und trägt nur die Programme
Selbstzerstörungs-PIN mit Vollwischmodus	OnlyKey	**teilweise.** Selbstlöschung nach Fehlversuchen ist zugesagt, eine eigene Wisch-PIN nicht
Glaubhafte Abstreitbarkeit — zweites Profil für den Zwangsfall	OnlyKey	**fehlt.** Unsere vier Profile sind getrennt, aber nicht abstreitbar. Das Bedrohungsmodell nennt „erzwungener Tastendruck" ausdrücklich als nicht abgedeckt
Fingerabdruck im Gerät	YubiKey Bio (5 Abdrücke, geteilte PIN für PIV und FIDO)	**in Stufe 1 nicht verbaut** — kein geprüfter Sensor erfüllte alle vier Anforderungen. Die Platine behält die Stiftreihe J5. Und `BioEnrollment` wird **bewusst abgelehnt**: kein Browser und keine Website darf einen Finger einlernen
Lightning für iPhones ohne NFC	YubiKey 5Ci	**fehlt**, und das bleibt so: proprietär, von USB-C abgelöst
RFC-3161-Zeitstempel	— (auch Yubico hat das nicht)	**im Plan oben zugesagt, in den Unterlagen nicht vorhanden.** Keine Zeitstempel-Fähigkeit irgendwo beschrieben — eine Zusage ohne Entsprechung

3. Was aus dem YubiHSM 2 übernehmenswert ist — getrennt betrachtet

Das YubiHSM 2 ist ein **Server-Modul**, kein Schlüssel: keine Anzeige, kein Taster, keine Benutzerbestätigung. Es löst ein anderes Problem. Vier seiner Eigenschaften treffen aber unseren Dongle- und Server-Anwendungsfall (NGC):

Eigenschaft	Was sie leistet	Stand bei uns
16 Rechtebereiche mit Fähigkeiten und übertragbaren Fähigkeiten — Rollen entstehen aus Kombinationen, es gibt keinen „Administrator"	Mehrere Dienste teilen ein Gerät, ohne einander zu erreichen; ein Betreiber kann keinen mächtigeren Schlüssel anlegen als er selbst hat	**fehlt vollständig.** Unser Entwurf kennt nur eine PIN und Profile
Mehrere Authentifizierungsschlüssel gleichzeitig, jeder mit eigenem Passwort oder ECP256-Schlüssel	Jede Software hat eigene Zugangsdaten, einzeln entziehbar	**fehlt**
Fälschungssicheres Protokoll im Gerät — jede Zeile mit der vorigen hash-verkettet, Änderungen und Löschungen werden erkennbar	Ein Protokoll, dem man ohne Vertrauen in den Server glauben kann	**teilweise.** Wir planen ein Protokoll, aber nicht verkettet
M-von-N-Schwellenwert auf dem Wrap-Schlüssel — z. B. 4 von 7 Verwaltern nötig, um einen Schlüssel auf ein neues Gerät zu holen	Ein Wurzelzertifikat übersteht den Verlust des Geräts, ohne dass einer allein es mitnehmen kann	**teilweise.** Shamir ist bei uns nur für das Hauptkennwort geplant, nicht für Schlüssel

Beim Schlüssel-Wrapping ist es keine Entscheidung, sondern eine Chip-Grenze — und das ist der wichtigste Einzelbefund dieses Abgleichs. Yubico exportiert Schlüssel mit AES-CCM-Wrapping aus dem HSM heraus, mit M-von-N darauf. Unser Sicherheitschip **kann das nicht**, und das ist im Repo schon ausgemessen und als Irrweg protokolliert, damit es niemand erneut versucht: `ExportObject` kann es nicht, `ReadObject` verweigert private und symmetrische Schlüssel, der Chip umhüllt nur *transiente* Objekte mit eigenen, nicht wählbaren Schlüsseln und verweigert den Import, solange das Original noch existiert (AN12413 3.2.8, 4.7.3.2, 4.7.1.8 — `docs/architecture.md` „Two key modes"). Eine frühere Zusage, „ein Wurzelgeheimnis im Chip, alles daraus im Chip abgeleitet", war darauf unmöglich und **ist bereits gestrichen**.

Die Folge ist unangenehm und gehört offen ins Bedrohungsmodell: **ein M-von-N- Verfahren für Schlüssel ist auf diesem Chip nicht baubar.** Was geht, ist das vorhandene Shamir-Verfahren auf dem **Dateischlüssel** der Sicherungsdatei (3 von 5 und ähnliches, `osk_shamir.c`) — ein echter Schwellenwert, aber er schützt eine Datei, nicht den Chipschlüssel. Wer ein Wurzelzertifikat will, das den Geräteverlust übersteht, braucht einen anderen Chip. **Das ist eine Entscheidung für Thomas, keine Aufgabe für einen Strang.**

4. Zertifizierungen — Standard einhalten, Papier schieben

> **Arbeitsregel von Thomas, 26.10.2026, und sie gilt dauerhaft für alle Projekte, nicht nur hier:** „Wir wollen technisch so sicher sein, aber noch keine Zertifizierungen machen. Aber diese machen können, wenn wir wollen. **Das gilt immer.** Wenn ich dir sage, wir wollen das und das haben, dann gilt immer: **den Standard einhalten, was gefordert ist.** Die Zertifizierung schieben wir."

Das ersetzt die ursprüngliche Haltung „nur benennen". **Der Unterschied ist groß:** Nicht benennen und weglassen, sondern **die Anforderungen des Standards bauen** und nur den bezahlten Prüflauf aufschieben. Und dabei nichts tun, was eine späte Zertifizierung verbaut.

Für FIPS 140-3 Level 3 heißt das konkret (eine eigene Sitzung arbeitet es gerade zu einer Bauliste aus):

Gemessen und belegt am 26.10.2026 — vier Annahmen waren falsch, alle zu unserem Vorteil:

Vermutet	Belegt
Level 3 braucht eine Manipulations-reaktion, also kommt der Akku zurück	**Falsch.** Die Reaktionspflicht gilt nur, wenn das Modul Türen, abnehmbare Deckel oder eine Wartungsschnittstelle hat, und nur solange Klartext-Schlüssel darin liegen. **Der Akku bleibt gestrichen**
Level 3 in Stickform ist kaum machbar	**Falsch.** Der **iStorage datAshur PRO** (NIST Security Policy 140sp4918) ist ein USB-Stick am Schlüsselring mit **FIPS 140-3 Overall Level 3 — und ohne jede Tamper-Schaltung.** Nur Serienbauteile, hartes undurchsichtiges Gehäuse mit Epoxid-Verguss, Auslesesperre
Der FIPS-SE051 verweigert ECDH	**Falsch** — und das berichtigt `docs/se050.md`. ECDH über **P-256 ist geprüft** (CVL C1820, KAS-SSC). Es fehlt nur ECDH über Curve25519. Dass EdDSA und Curve25519 fehlen, stimmt
Der Tresor muss AES im Rechenkern rechnen, weil der Chip es nicht kann	**Falsch.** AES-256 GCM/GMAC ist im Chip geprüft. Der Tresor im Rechenkern ist eine **Durchsatz**-Entscheidung, keine Fähigkeitslücke

Und der Widerspruch, den ich benannt hatte, dreht sich: Bei einer Modulgrenze **um das Gehäuse** (multi-chip standalone) ist **ML-KEM im Rechenkern möglich** — Prüfverfahren für FIPS 203 existieren. Unmöglich ist es nur bei einer Grenze um den Sicherheitschip allein, denn SE051C2 und SE052F können keine Post-Quanten-Verfahren. **Der Konflikt heißt also „ML-KEM gegen die Chip-Grenze", nicht „ML-KEM gegen FIPS".** Empfehlung des Strangs: Grenze um das Gehäuse.

Was daraus als Pflicht folgt, und drei Punkte müssen jetzt in die Platine:

1. **Epoxid-Verguss** über allen sicherheitsrelevanten Bauteilen — er ersetzt die Tamper-Schaltung. Platz, Höhe, Wärmeabfuhr und ein Fertigungsschritt gehören eingeplant; Programmier- und Prüfpunkte müssen danach unerreichbar sein. **Später nicht nachrüstbar.**
2. **Kein abnehmbarer Deckel**, kein Zugang zur Baugruppe.
3. **Keine Wartungs- oder Debug-Schnittstelle definieren** — allein ihre Existenz löst die Pflicht zur Reaktionsschaltung aus, und damit käme der Akku zurück.

Dazu drei Punkte für die Firmware:

- **Die PIN darf nie über USB laufen.** Ab Stufe 3 ist ein vertrauenswürdiger Kanal Pflicht. Taster und Anzeige haben wir — **damit wird die PIN-Eingabe am Gerät aus Abschnitt 1 von einer guten Idee zur Anforderung**, und der Weg über den Rechner darf gar nicht erst existieren.
- **Nie ein Klartextschlüssel in Ruhe.** Kette: PIN → langsame Ableitung → KEK → entschlüsselt DEK. Sobald irgendwo ein Klartextschlüssel im Flash liegt, kommt der Akku zurück.
- **Ed25519 und X25519 nur als abschaltbare Nicht-FIPS-Erweiterung**, sonst braucht es später eine Zustandsumschaltung im Gerät.

Level 4, nur technisch betrachtet: als Geräte-Eigenschaft **nicht baubar** — eine Erkennungshülle mit Reaktion über die ganze Baugruppe braucht Dauerstrom und ist in 0,8 mm mit Kantenstecker nicht unterzubringen; dazu kommen Umweltschutz als Pflicht und mehrfaktorige Authentifizierung, die mit einem Taster und 0,71 Zoll nicht zu erfüllen ist. **Als Chip-Eigenschaft ist physische Sicherheit Stufe 4 dagegen schon erreicht** (SE051/SE052F, aktives Schirmgitter, ohne eigene Energiequelle).

Zur Einordnung der Messlatte: **Der YubiKey 5 FIPS erreicht Overall Level 2 mit Physical Security Level 3.** Overall Level 3 hat bei Yubico nur das YubiHSM. Unser Vorbild ist also nicht Yubico, sondern der datAshur PRO — ein Stick, der Overall Level 3 mit Verguss statt mit Elektronik erreicht.

Die vollständige Auswertung mit den elf Anforderungsbereichen als Bauliste und **21 nummerierten Bauaufgaben** liegt in `D:\claude\side-session-notes\fips-140-3.md`. Einschränkung, die dazugehört: ISO 19790 ist kostenpflichtig, die Stufenaussagen stützen sich auf NIST-Veröffentlichungen, FIPS 140-2 und veröffentlichte Security Policies — in der Notiz jeweils als `ungeprueft` gekennzeichnet.

Und der Punkt, der bei Zertifizierungen am häufigsten übersehen wird: Zertifizierung und laufende Entwicklung passen nicht zusammen. Jede Firmware-Änderung verlangt einen neuen Prüflauf — darum hat Yubico eine eigene FIPS-Produktlinie mit eingefrorener Firmware. Wer später zertifizieren will, braucht also **einen eingefrorenen Zweig**, nicht nur ein erfülltes Regelwerk. Das ist eine Entscheidung über die Veröffentlichungsform, und sie gehört jetzt notiert, weil sie später teuer wird.

Die Papiere selbst, zur Übersicht

Zertifizierung	Bei Yubico	Was sie kostet
FIPS 140-3, Overall Stufe 2, Physical Security Stufe 3	YubiKey 5 FIPS (Zertifikat #5291, Firmware 5.7.4)	Prüflabor, Monate, fünfstellig
FIPS 140-3 Overall Stufe 3	YubiHSM 2 FIPS (Zertifikat #5302)	mehr
CCN ENS Alta (Hoch)	YubiKey 5.7.4	spanische Behördenprüfung
CSPN (ANSSI)	YubiKey 5.4.2, eingestellt	—
FIDO-Zertifizierung	alle FIDO-Produkte	Gebühr, und sie **verträgt sich nicht mit gespiegelten Passkeys** (offener Punkt 4 oben)

Das sind Papiere, keine Technik. **Keine davon wird jetzt beantragt** — aber nach der Arbeitsregel oben werden ihre technischen Anforderungen eingehalten, und es wird nichts getan, was eine späte Beantragung verbaut. Der Sicherheitschip ist mit EAL6+ geprüft, was dieselbe Angriffsklasse abdeckt wie die physischen Anforderungen von FIPS.

Ein Gegensatz bleibt und muss entschieden werden: Eine FIDO-Zertifizierung verträgt sich nicht mit gespiegelten Passkeys (offener Punkt 4 oben). Hier lässt sich nicht beides einhalten — entweder das Zweitgerät ist ein echter Ersatz, oder die Bescheinigung ist gerätegebunden. Das ist der eine Fall, in dem die Arbeitsregel nicht greift, weil zwei Anforderungen einander ausschließen.

5. Was wir können und Yubico nicht — zur Einordnung

Damit die Liste oben nicht den Eindruck hinterlässt, wir hinkten hinterher:

- **Echter Passwortspeicher** — Yubico hat keinen. OnlyKey schafft 24 Konten, Nitrokey 3 mehr, Nitrokey Pro/Storage nur 16. Wir sagen 768 je Profil zu, bei vier Profilen 3072 — zwei Größenordnungen mehr als jeder Mitbewerber.
- **Krypto-Geldbörse** mit Anzeige von Empfänger und Betrag — bei keinem der verglichenen Schlüssel vorhanden. **Berichtigung zum Plan oben:** die Unterlagen sagen **nur Bitcoin und Ethereum** zu (`docs/wallet.md`). Solana und Cardano stehen oben als Zusage, sind aber nirgends umgesetzt — Ed25519 kommt nur als Kommentarwort in einem Header vor. Auch Taproot wird ausdrücklich abgelehnt, „die Hälfte davon wäre gefährlicher als nichts".
- **Dongle-Betrieb** mit Entschlüsselung geschützter Programmteile — gibt es nur bei Yubico als HSM, nicht als Schlüssel.
- **Anzeige am Gerät** — kein Yubico-Produkt hat eine. Sie ist die Grundlage dafür, dass man dem Rechner nicht glauben muss.
- **Bluetooth** für das eigene Programm, **Stick-zu-Stick-Übertragung** ohne Wolke, **Massenspeicher mit portablen Programmen** — alles bei Yubico nicht vorhanden.

6. Der Nebenbefund, der dringender ist als der Marktvergleich

Beim Abgleich hat sich gezeigt: **die eigenen Unterlagen widersprechen sich an acht Stellen.** Das ist kein Schönheitsfehler — solange sie es tun, ist jeder Vergleich mit einem Fremdprodukt auf Sand gebaut, weil nicht feststeht, was wir selbst haben. Die vier schwerwiegenden:

1. **Ist für ARM übersetzt worden?** `docs/build.md` und `docs/architecture.md` sagen ja, mit gemessenen Zahlen (161 124 B Flash, 25.10.2026). `docs/update.md` sagt „nichts ist je für ARM übersetzt worden", und `CHANGELOG.md` bestätigt das. **Eine der beiden Seiten ist falsch**, und davon hängt ab, auf welcher der vier Stufen von „geprüft" das Projekt steht.
2. **Ist PIV, OpenPGP, Wallet, Anzeige, Funk geschrieben?** `docs/architecture.md` sagt „nichts davon ist geschrieben". Die vier Einzeldokumente beschreiben alles als geschrieben, übersetzt und am PC getestet, mit Dateilisten.
3. **Wie viele Credentials?** `docs/ctap.md` sagt „unbegrenzt", weil abgeleitet statt gespeichert wird. `docs/architecture.md` sagt, im Modus „nur dieses Gerät" belege jedes Credential Chipspeicher, die Zahl sei also endlich. **Beides kann nicht stimmen**, und es ist genau die Zahl, die Yubico mit 100 Passkeys beziffert.
4. **Nutzerspeicher des Chips: 46, 50 oder 104 kB?** Drei Dokumente, drei Zahlen. Darauf ruht die Entscheidung, Schlüssel abzuleiten statt zu speichern.

Dazu vier kleinere: ob die Auslagerung des Update-Platzes nötig war, 27,5 gegen 29,10 MB für die portablen Programme, ob PKCS#11 noch jeden Schlüssel einzeln freigeben lässt, und — am folgenreichsten — **ob der gelieferte Chip Ed25519, X25519 und ECDH überhaupt kann**. Die FIPS-Politik eines echten SE051 zeigt die Merkmalsmaske `0x27F2`: EdDSA aus, Curve25519 aus, ECDH verweigert. Die Variantentabelle AN12973 sagt das Gegenteil. Entschieden wird es erst am gelieferten Teil — und wenn die Maske gilt, **haben SSH-Schlüssel, Teile der Geldbörse und die Server-Schlüsseleinigung keinen Boden.**

Das ist sechsmal dasselbe Muster wie an der Platine: **zwei Größen, die gleich heißen und Verschiedenes messen.** Es gehört vor den Marktvergleich geräumt, nicht danach.

7. Brauchen wir andere Hardware? — geprüft, und die Antwort ist dreiteilig

Thomas' Vermutung lag nahe. Sie trifft aber nur einen kleinen Teil der Lücken. **Von den Fähigkeiten in Abschnitt 2 und 3 ist die große Mehrheit Software.**

7a. Was gar keine neue Hardware braucht

Lücke	Warum sie Software ist
RSA	Die Merkmalsmaske eines echten SE051 hat **RSA plain und RSA CRT an** (`docs/se050.md`). Der Chip kann es — **unser Treiber verweigert es**, als bewusste Entscheidung. RSA ist also eine Entscheidung, keine Hardware-Grenze. **Thomas hat entschieden: die Hardware soll beides können, und es gibt zwei Firmware-Varianten** — siehe 7f
CredentialManagement (0x0A), Config (0x0D), Enterprise Attestation	Firmware
Yubico-HID-Transport (und damit KeePassXC, VeraCrypt)	Firmware. Es fehlt eine befolgbare Beschreibung, nicht ein Chip
Signaturzähler in den Flash	Firmware
PIN-Eingabe am Gerät	**Die Hardware ist schon da** — Anzeige plus Taster
Verschlüsselter Massenspeicher, Abstreitbarkeit, Wisch-PIN, PIN-Regelwerk	Firmware
Verkettetes Protokoll, mehrere Zugangsschlüssel, Rechtebereiche	Firmware

7b. Was eine Beschaffungsentscheidung ist, keine neue Hardware

Das ist der gefährlichste Punkt des ganzen Abgleichs. EdDSA, Ed25519 und Curve25519 gibt es laut NXPs Konfigurationsschrift AN12973 **nur bei der Variante SE051C2** — und genau die ist gewählt. Aber die Merkmalsmaske wird **im Werk je Konfiguration gesetzt und ist danach nicht änderbar**, und die FIPS-Konfiguration hat EdDSA und Curve25519 **aus** und verweigert ECDH.

> **Wird das falsche Teil bestellt, haben SSH-Schlüssel, Teile der Geldbörse und die Server-Schlüsseleinigung keinen Boden** — und man merkt es erst am gelieferten Chip. Das gehört **vor** die Bestellung, nicht in eine Prüfstufe.

Dazu passt ein zweiter Fund: NXP-Module lassen sich über das Perso-Applet abspecken, **EdDSA ausdrücklich eingeschlossen**, um Speicher zu gewinnen. Ein Chip kann die Fähigkeit also auch verloren haben.

7c. Was wirklich andere Hardware bräuchte — und was die Alternativen taugen

Nur **zwei** Lücken sind echte Hardware-Grenzen: **Schlüsselexport mit Wrapping** (und damit M-von-N für Schlüssel, Abschnitt 3) und der **Fingerabdrucksensor** (für den die Platine die Stiftreihe J5 behält).

Die naheliegenden Alternativen sind **schlechter**, nicht besser:

Chip	Was er kann	Urteil
Infineon OPTIGA Trust M	NIST P-256/P-384, RSA 1024/2048, 10 kB Nutzspeicher, CC EAL6+, als Adafruit-Breakout frei zu kaufen	**kein secp256k1, kein Ed25519, ein Zehntel des Speichers.** Geldbörse und SSH fielen weg
Microchip ATECC608	P-256	**deutlich schwächer**
JavaCard / JCOP 4.5 (z. B. J3R452, P71, 450 KB)	eigene Applets **im Sicherheitschip**, alle Kurven, RSA, GlobalPlatform-Schlüsselverwaltung mit eigenen Regeln, monotone Zähler, 450 kB statt 50–104	**der einzige Weg, der wirklich besser ist** — und genau der Weg, den Yubico geht. Preis: Java-Werkzeugkette (Oracle JDK, JCOP-Plugin, GPShell), und die Beschaffung läuft über Reseller und Dienstleister; **ob NXP das SDK ohne Geschäftskonto herausgibt, ist** `ungeprueft`

7d. Die Pointe: Yubicos Hardware war die unsicherere

Yubico nutzt den Infineon SLE78 — und dessen ECDSA-Bibliothek hatte eine nicht-zeitkonstante modulare Inversion. Der Angriff **EUCLEAK** (NinjaLab, 2024) holt damit bei physischem Zugang in wenigen Minuten elektromagnetischer Messung den ECDSA-Schlüssel heraus. **Jeder YubiKey 5 vor Firmware 5.7 war klonbar.** Unser SE051 war davon nicht betroffen.

Das heißt nicht, dass unser Chip besser ist — es heißt, dass „andere Hardware" kein Qualitätsbeweis ist. Und es ist ein Argument für etwas, das im Entwurf oben schon steht und jetzt Gewicht bekommt: **Vergleiche mit gleicher Laufzeit und Zufallsverzögerung** im eigenen Code, denn genau diese Klasse Fehler hat den Marktführer getroffen.

7f. Entschieden: die Hardware kann beides, die Firmware kommt zweimal

Thomas' Entscheidung: Der Chip soll RSA *und* die Edwards-Kurven können, und es gibt **zwei Firmware-Varianten** — eine mit RSA, eine ohne. Wer die schlanke Fassung will, bekommt sie; wer PIV in einer Firmenumgebung braucht, die andere.

Das ist mit Zephyr sauber baubar (eine Kconfig-Option, zwei Abbilder, wie die Sprachdateien). **Aber es hat vier Folgen, die in den Plan gehören:**

1. **Es verschärft die Bestellfrage aus 7b, und zwar an der kritischen Stelle.** Die einzige Merkmalsmaske, die in den Unterlagen belegt ist — `0x27F2` aus der FIPS-Sicherheitspolitik — hat **RSA an und EdDSA aus**. Genau die Kombination, die wir *nicht* wollen. **Ob es überhaupt eine Konfiguration gibt, die RSA, EdDSA, Curve25519 und ECDH zusammen anbietet, ist offen** und muss die Antwort von Schritt 0 sein. Findet sich keine, ist Thomas' Wunsch mit diesem Chip nicht erfüllbar — und dann wird JavaCard (7c) aus einer Möglichkeit eine Notwendigkeit.

2. **Zwei Abbilder heißen zwei Signaturen, zwei Prüfläufe, zwei Veröffentlichungen.** Der reproduzierbare Bau und die Prüfsummen gelten je Variante. Die Werkzeuge in `tools/` müssen die Variante im Dateinamen und im Abbildkopf führen, sonst spielt jemand die falsche auf.
3. **Zwei AAGUIDs.** FIDO verlangt, dass ein AAGUID eine Gerätefassung mit *denselben* Fähigkeiten bezeichnet. Zwei Varianten mit verschiedenem Algorithmenumfang sind zwei Fassungen. Der AAGUID ist ohnehin noch vorläufig (`docs/ctap.md`) — das ist der Moment, zwei vorzusehen statt später zu teilen.
4. **Der Chipspeicher wird knapp.** RSA-4096-Schlüssel sind groß, und der Nutzspeicher ist 50 oder 104 kB (die Zahl ist einer der acht Widersprüche). Wie viele RSA-Schlüssel hineinpassen, ist zu rechnen, bevor Kapazitäten zugesagt werden.

7g. Was daraus folgt

1. **Die Chip-Bestellnummer festnageln** (7b und 7f) — dringend, kostet nichts, und sie entscheidet inzwischen mehr als nur eine Kurve.
2. **RSA als Baueinstellung** mit zwei Abbildern, wie entschieden, samt den vier Folgen oben.
3. **M-von-N auf Schlüsseln bleibt ungebaut**, solange der Chip derselbe ist. Das gehört so ins Bedrohungsmodell.
4. **JavaCard wird geprüft** (Thomas' Entscheidung): eigener Strang, der Beschaffbarkeit ohne Geschäftskonto, Werkzeugkette, Aufwand und den Verbleib des bisherigen Codes klärt. Ergebnis ist eine Entscheidungsvorlage, kein Umbau. Sie verlagerte die Sicherheitslogik vom Rechenkern in den Chip und machte den nRF52840 zum Vermittler — eine Entscheidung über das halbe Projekt. **Und sie hängt an Punkt 1:** findet sich keine Chip-Konfiguration mit RSA und Edwards-Kurven zusammen, ist JavaCard der einzige Weg zu Thomas' Wunsch.

8. Verschlüsselung, Signatur, Zertifikate, Passwörter — die größeren Lücken

Thomas hat nach diesen Bereichen gefragt, und der Befund ist unangenehmer als der Marktvergleich: **drei Dinge, die der Entwurf oben ausdrücklich zusagt, existieren nirgends.** Nicht als Lücke benannt, nicht als offener Punkt — gar nicht. Alles unten mit Belegstelle gemessen, nicht erinnert.

8a. Dateien ver- und entschlüsseln: es gibt nichts

Der Entwurf oben sagt zu: „Dateien verschlüsseln und entschlüsseln, mit Zeitstempel signieren". **Gefunden wurde kein einziger Weg dafür.**

- **Kein S/MIME, kein CMS, kein PKCS#7, kein OpenPGP-Nachrichtenformat.** OpenPGP gibt es nur als *Kartenanwendung* (`docs/smartcard.md`) — das Gerät kann als Smartcard signieren und entschlüsseln, aber es kennt kein Nachrichtenformat.
- **Kein Befehl.** Die Kommandozeile kennt `get/keys/put/env/run/log/codes`, `unlock`, `guard-key`, `guard-agent` — **kein** `encrypt`**, kein** `decrypt` (`docs/anbindungen.md`).
- Das Einzige, was verschlüsselt, ist das Dongle-Siegel `OSKSEAL1` (`shared/osk-guard/README.md`) — eine Bibliothek für Programme, kein Dateiwerkzeug. **Und es hat eine harte Grenze: 4 MiB je Protokollzeile, alles als Hex, ein Vorgang je Zeile.** Eine 2-MB-Datei füllt die Zeile voll. Es gibt kein Blockverfahren für Nutzdateien — nur für die Tresor-Sicherung und für Firmware-Updates.

Entschieden (Thomas): beide Formate. OpenPGP für E-Mail und Verträglichkeit, `age` **für Dateien**. age passt außergewöhnlich gut: X25519 und ChaCha20-Poly1305 — X25519 haben wir am Chip —, eine Erweiterungsschnittstelle für Hardware-Schlüssel (`age-plugin-yubikey` ist das Vorbild), und **seit 2025 post-quanten-hybride Empfängertypen im Format selbst**. Damit kommt 8e fast geschenkt dazu.

Zu lösen ist dabei das Blockverfahren, nicht das Format: ein Dateiwerkzeug ohne Streaming ist bei der ersten Videodatei am Ende.

8b. Zertifikate: der Stick kann sie vorzeigen, nicht beantragen

Der Entwurf oben sagt zu: „Eigene Zertifizierungsstelle im Gerät: eigene CA erzeugen und damit Zertifikate für eigene Geräte und Dienste ausstellen."

Gefunden: kein PKCS#10, kein CSR, keine Kette, kein Widerruf. Das Repo sagt es selbst: „No certificate is created yet… Creating one needs the firmware side first" (`docs/desktop.md:290`).

Fehlt	Folge
CSR (PKCS#10) auf dem Gerät	Ohne Antrag gibt es kein Zertifikat zu einem Schlüssel, der den Chip nie verlässt. **Das ist die Grundfunktion**, ohne die PIV- und S/MIME-Zertifikate nicht entstehen können
Zertifikatskette, Zwischenzertifikate	Eine eigene CA ohne Kette ist keine CA
CRL oder OCSP	**Eine CA ohne Widerrufsweg ist unverantwortlich.** Wer ein Zertifikat ausstellt, muss es zurückziehen können
X.509-Erweiterungen (SAN, Key Usage, EKU)	Ohne SAN akzeptiert kein Browser ein Serverzertifikat
Ablaufwarnung	Niemand merkt, dass ein Zertifikat ausläuft

Nebenbefund zur Signatur: **RFC-3161-Zeitstempel fehlt**, und damit auch die PAdES-Stufen B-T und B-LT — die PDF-Signatur, die über die einfachste Stufe hinausgeht, braucht eine Zeitstempelstelle. Ebenso fehlen **PAdES/CAdES/XAdES ganz**, also jede PDF-Signatur, sowie `sshsig` und **Git-Commit-Signatur** als eigene Funktionen — beide stehen oben als Zusage. Was es gibt, ist Authentifizierung über den ssh-agent, und das ist etwas anderes als eine Dateisignatur.

8c. Passwörter: man kommt nicht hinein und nicht heraus

- **Kein einziger Importweg.** KeePass und CSV stehen oben als Zusage — **gefunden wurde nichts**, weder KeePass, CSV, 1Password, Bitwarden, LastPass noch Browser-Export oder QR-Code. Der einzige Eingabeweg ist `osk put` für *einen* Eintrag.
- **Keine Leak-Prüfung** (HIBP, k-Anonymität) und **keine Stärkeprüfung** für gespeicherte Passwörter. Oben zugesagt ist „doppelte, zu kurze und zu alte Passwörter finden".
- **Kein Passkey-Export**, und das Repo nennt die Folge selbst: ob ein neuer Stick aus der Sicherung die Passkeys wieder nutzen kann, ist „nicht festgelegt" (`docs/en/troubleshooting.md:35`). Dazu fehlen die Befehle zum Auflisten und Löschen von Passkeys ganz — dieselbe Lücke wie `authenticatorCredentialManagement` in Abschnitt 2a, von der anderen Seite.

Entschieden (Thomas): CXF aufnehmen, Import und Export. Das **Credential Exchange Format** der FIDO Alliance ist seit August 2025 verabschiedeter Standard und deckt **Passkeys, Passwörter, TOTP und SSH-Schlüssel** ab; Apple und Google haben Schnittstellen dafür, 1Password hat mitgeschrieben. Der Transport heißt CXP und baut einen gemeinsamen Schlüssel per Diffie-Hellman, damit nur der gedachte Empfänger entschlüsseln kann.

Das schlägt zwei Lücken mit einem Schlag: der Importweg aus fremden Managern **und** der Ausweg, der heute „nicht festgelegt" ist. Es ist zugleich der ehrlichere Weg als eine Einbahnstraße — genau das, was Nutzer an anderen Anbietern stört.

Eine Spannung dazu gehört ins Bedrohungsmodell: Export heißt, dass Zugangsdaten das Gerät verlassen können. Das widerspricht „Schlüssel verlassen den Chip nie" — und ist für Passkeys ohnehin schon entschieden (der Modus „gespiegelt" existiert oben). CXF macht es nur sichtbar und muss dieselbe Freigabe verlangen: PIN, Tastendruck, und eine Anzeige, die sagt, **wohin**.

Ein Lob an den Entwurf an einer Stelle, die man leicht falsch machte: **die Geräte-PIN nutzt absichtlich kein langsames Verfahren** — `HKDF` statt Argon2 — und die Begründung steht dabei: der Schutz kommt aus dem Fehlversuchszähler im Chip, nicht aus Rechenkosten (`docs/se050.md:373`). Für die Sicherungsdatei ist es umgekehrt Argon2id, auf dem Stick 64 KiB/64 Durchgänge, am Rechner 64 MiB/3. Das ist beides richtig und sauber getrennt.

8d. Ordner und Laufwerke: zwei von drei fallen uns zu

Thomas hat ausdrücklich nach **Ordner-, Datei- und Laufwerks**verschlüsselung gefragt. Das sind drei verschiedene Dinge, und sie stehen völlig verschieden — die Antwort ist überwiegend erfreulich.

	Stand	Was zu tun ist
Datei	**fehlte ganz** (8a)	bauen: `age` und OpenPGP-Nachrichten, mit Blockverfahren
Ordner	**fällt uns zu**	nur prüfen und dokumentieren
Laufwerk (Linux, LUKS2)	**fällt uns zu**	nur prüfen und dokumentieren
Laufwerk (Windows, BitLocker)	`ungeprueft`	messen
VeraCrypt	zwei Wege, einer offen	siehe unten
Eigenes Container-Format	nicht geplant — **und soll es bleiben**	nichts

Der wichtigste Befund dieses Abschnitts: Für LUKS2 über `systemd-cryptenroll` muss ein Gerät **nur die FIDO2-Erweiterung hmac-secret beherrschen** — plus PIN und Tastendruck, die beide voreingestellt sind. **Und hmac-secret ist bei uns fertig** (`docs/ctap.md`: done). Es braucht von uns keine Festplattenverschlüsselung, keine Adminrechte und keine OS-Integration: systemd macht die Arbeit, wir sind nur der Schlüssel.

Dasselbe bei Ordnern: `gocryptfs` **hat eine** `-fido2`**-Option**, und ein Nitrokey-Passkey — ein Gerät mit *weniger* Fähigkeiten als unseres — arbeitet damit nachweislich. Auch hier genügt hmac-secret.

Das berichtigt eine Einordnung in den Unterlagen. Dort steht „Festplattenentsperrung und PAM-Anmeldung: absichtlich nicht gebaut — alles OS-Integration mit Administratorrechten" (`docs/anbindungen.md`). **Für LUKS und gocryptfs ist das zu pessimistisch**: dort ist nichts zu bauen. Für PAM und den Windows-Minidriver bleibt es richtig.

Was daraus folgt — und es ist weniger Arbeit, nicht mehr:

1. **Nichts bauen, sondern messen und ins Handbuch schreiben.** Je Weg ein echter Vorgang: ein LUKS2-Datenträger mit unserem Stick entsperrt, ein gocryptfs-Ordner damit geöffnet. Das sind zwei Messungen für Stufe 3 oder 8, keine Entwicklungsaufgaben. **Bis sie gelaufen sind, bleibt es** `ungeprueft` — dass die Erweiterung umgesetzt ist, heißt nicht, dass `libfido2` unser Gerät annimmt.
2. **Kein eigenes Container-Format bauen.** gocryptfs und Cryptomator existieren, sind offen und brauchen uns nur als Schlüssel. Ein eigenes wäre doppelte Arbeit an der gefährlichsten Stelle.
3. **VeraCrypt hat zwei Wege, und wir haben den besseren schon:** über **PKCS#11** (unsere Bibliothek ist gebaut) oder über Yubico-Challenge-Response (deren Transport fehlt, Abschnitt 2a). Der PKCS#11-Weg ist zu messen, bevor die Challenge-Response-Lücke als Blocker geführt wird — womöglich ist sie für VeraCrypt keiner.
4. **BitLocker** nutzt denselben Mechanismus unter anderem Namen (PRF über hmac-secret). Unsere Unterlagen behaupten das schon (`docs/ctap.md` nennt BitLocker und LUKS als hmac-secret-Nutzer) — **belegt ist es nicht**, und für Windows fand die Recherche keine Quelle. Also messen, nicht behaupten.

8e. Post-Quanten: im Repo steht nicht ein Wort

Kein Treffer für PQC, ML-KEM, ML-DSA, SLH-DSA, Kyber, Dilithium — und auch nicht als offener Punkt oder begründete Weglassung. Der einzige Treffer für „quantum" im ganzen Repo ist ein BIP39-Wörterbuchwort.

Die NIST-Standards sind seit **August 2024** fertig, die COSE-Registry hat die Verfahren seit April 2025, FIDO arbeitet daran — und **kein Hardware-Schlüssel am Markt hat sie**, Yubico eingeschlossen. Der Sicherheitschip kann sie nicht, und seine Kurvenwahl ist fest; eine PQC-Erweiterung im Chip ist damit ausgeschlossen.

Entschieden (Thomas): hybrid einplanen. ML-KEM **zusammen mit** X25519, gerechnet im Rechenkern. Betroffen sind die Stellen, an denen Verkehr abgefangen und später entschlüsselt werden könnte:

1. **Stick-zu-Stick-Übertragung** — der ganze Tresor wandert dort hinüber
2. **Server-zu-Server** (NGC) — X25519-Sitzungsschlüssel
3. **CXP-Export** (8c), der ebenfalls Diffie-Hellman nutzt
4. **age** bringt es im Format schon mit

Nicht betroffen: die Sicherungsdatei. Sie ist symmetrisch (Argon2id + AES-256-GCM), und AES-256 gilt gegen Quantenrechner als ausreichend. Das ist eine gute Nachricht und gehört dazugesagt, damit niemand die Datei unnötig umbaut.

Der Preis, der ausgesprochen werden muss: Ein ML-KEM-Schlüssel, der im Rechenkern gerechnet wird, liegt **nicht** im Sicherheitschip. Für die hybride Konstruktion ist das vertretbar — fällt der PQC-Teil, bleibt X25519 aus dem Chip und die Sicherheit ist nicht schlechter als heute. **Umgekehrt gilt es nicht:** ein reiner PQC-Pfad ohne X25519 wäre ein Rückschritt. Das gehört als Regel in das Bedrohungsmodell, nicht als Fußnote.

9. Die Hardware-Entscheidung

Thomas hat um eine Entscheidung gebeten, nicht um eine Frage. Hier ist sie, mit den drei Messgrößen, auf denen sie ruht.

9a. Was gemessen wurde

Größe	Wert	Quelle
USB-Durchsatz des nRF52840	**400–500 kB/s**, nicht 1,5 MB/s. Bulk-Endpunkte sind auf 64 Byte je Mikroframe begrenzt — 500 kB/s theoretisch, mit Dateisystem gemessen **400 kB/s**	Nordic selbst im DevZone, dazu eine Messung am nRF5340 mit demselben USB-Baustein. `ungeprueft` für unseren Aufbau, aber zweifach belegt
ML-KEM-768 auf Cortex-M4	~3,5 Mio Zyklen in der **unoptimierten** Referenz = **55 ms bei 64 MHz**; Flash ~11 kB, Spitzen-RAM ~20 kB	pqm4-Zahlen über eine Cortex-M0+-Arbeit; die optimierten M4-Fassungen sind deutlich schneller. Größenangaben aus Fremdplattformen, also `ungeprueft`
Monotone Zähler im Sicherheitschip	**vorhanden** — „can only be incremented and not be decremented", 1 bis 8 Byte, dauerhaft oder flüchtig	NXP AN13187

9b. Die Entscheidung: kein Wechsel — mit einer Streichung und zwei Bedingungen

Der nRF52840 und der SE051C2 bleiben. Begründung, Punkt für Punkt:

- **Post-Quanten passt mühelos.** 55 ms einmal je Übertragung ist nichts, und 11 kB Flash sind bei 157 von 1024 kB Belegung kein Thema. Thomas' Entscheidung aus 8e ist auf dieser Hardware umsetzbar.
- **Der Rückdrehschutz ist baubar.** Der Chip hat monotone Zähler. Die beiden fehlenden Aufrufe `osk_se050_counter_read()` und `counter_raise()` sind Software, und sie blockieren heute schon jede Paketprüfung des Bootladers (`docs/update.md`). **Das war als Hardware-Grenze verdächtigt und ist keine.**
- **RSA kann der Chip**, secp256k1 auch, und die Alternativen OPTIGA Trust M und ATECC608 sind schwächer (Abschnitt 7c).
- **Ein Wechsel würde die Funkzulassung neu aufwerfen.** Das vorzertifizierte Modul ist der Grund, dass die Platine kein Funkgerät-Prüfverfahren braucht.
- **Yubicos Hardware war die unsicherere** (7d). „Anderer Chip" ist kein Qualitätsbeweis.

Die Streichung: der verschlüsselte Massenspeicher entfällt — nicht verschoben, gestrichen. Bei 400 kB/s ist ein Datenlaufwerk unbenutzbar, und **das ändert in dieser Geräteklasse kein anderer Chip**: alle Kandidaten mit BLE *und* NFC haben USB Full Speed. Nitrokey Storage kann sich 64 GB erlauben, weil es kein Bluetooth, kein NFC und keine Anzeige hat. Wir haben uns gegen diesen Tausch entschieden, und das ist richtig — aber dann gehört die Folge ausgesprochen statt als Ausbaustufe geführt.

> **Ein neuer Befund, der in die Unterlagen gehört:** Die 27 MB portabler Programme auf dem schreibgeschützten Laufwerk brauchen bei 400 kB/s **etwa 68 Sekunden** zum Lesen. „Einstecken, starten, keine Installation" ist also einstecken, **über eine Minute warten**, starten. Das ist kein Fehler, aber es muss im Handbuch stehen, und es begrenzt, wie groß die Programme werden dürfen — die Reserve von 6,82 MB ist damit nicht nur eine Platz-, sondern eine Geduldsfrage.

Bedingung 1: die Chip-Bestellnummer (Schritt 0). Gibt es keine Konfiguration mit RSA **und** EdDSA **und** Curve25519 **und** ECDH zusammen, fällt diese Entscheidung — dann ist **JavaCard die Antwort**, nicht ein anderer Secure-Element-Hersteller. Die Entscheidung oben gilt unter dem Vorbehalt, dass diese Nummer existiert.

Bedingung 2: die Flash-Rechnung. Die ehrliche Hochrechnung liegt bei 264 kB von 960 (`docs/build.md`), und dazu kommen jetzt RSA, ML-KEM, age, OpenPGP- Nachrichten, CXF, CSR und die CA-Verwaltung. Das muss gerechnet werden, **bevor** gebaut wird. 1 MB ist viel, aber die Liste ist lang, und MCUboot braucht seinen Platz.

9c-neu. Nachtrag 26.10.2026: die Entscheidung ist knapper geworden

Nach einer zweiten, genaueren Runde haben sich **zwei Punkte gedreht**, und die Streichung des Massenspeichers aus 9b ist damit hinfällig — Thomas will ihn, und er ist baubar.

Was für einen Wechsel auf nRF54LM20A/B spricht, und in der ersten Runde fehlte:

Befund	Warum er zählt
NFC-A-Tag ist vorhanden	Das Ausschlusskriterium ist bestanden. Der Wechsel ist technisch überhaupt möglich
Das Modul ist kleiner: BL54LM20B 11 × 8 × 1,8 mm gegen MDBT50Q 15,5 × 10,5 × 2,05	Thomas' Hauptwunsch ist Kürze. Ob daraus Länge wird, muss der Platinen-Strang rechnen — die Flächenprobe sagt, das Bauteileband hängt an den Passiven, nicht am Chip
512 KB RAM statt 256	**Das ist das stärkste Einzelargument.** Die oberen 12 Ebenen des Merkle-Baums im Arbeitsspeicher zu halten kostet **128 KB**. In 512 KB ist das bequem, in 256 KB nur knapp. Ohne diesen Zwischenspeicher sind die 22 Pfadknoten wahlfreie 32-Byte-Zugriffe auf eine SD-Karte — **das Langsamste überhaupt, 0,14 MB/s**
2 MB Flash statt 1 MB	RSA, ML-KEM, age, OpenPGP-Nachrichten, CXF, CSR und die CA-Verwaltung müssen alle hinein (Bedingung 2 aus 9b). Mit 2 MB ist die Rechnung keine Zitterpartie
Zulassung bleibt beim Modul, Firmware bleibt größtenteils	Gleicher Hersteller, gleiches SDK

Was dagegen spricht:

Befund	Gewicht
Vorserie. Beta-Muster ab Oktober 2026, kein veröffentlichter Preis, Zulassungen teils „pending"	schwer, aber zeitlich begrenzt
Kein echtes QSPI — der heutige 64-MB-NOR-Flash liefe dort **langsamer als heute**	**Das ist der unangenehmste Befund.** Der Datenträger würde 7,5× schneller, der Tresor-Flash langsamer. Betroffen ist die Kernfunktion, nicht das Zubehör
Der Speicher bleibt die Wand: ~3 MB/s, nach **jedem** Wechsel	Faktor 7,5 statt 60. Aus unbenutzbar wird benutzbar, aber kein Datenträger im üblichen Sinn

Und ein Befund, der das Konzept betrifft, nicht die Hardware: Weil der Speicher mit 3 MB/s bremst und das Hashen 19–26 MB/s schafft, ist **der Unterschied zwischen Betriebsart 3 und den Betriebsarten 1 und 2 am Gerät gar nicht zu merken**. „Nur gesiegelt" ist nicht schneller als „verschlüsselt" — es sei denn, AES-GCM liegt unter 3 MB/s.

> **Die eine Zahl, die fehlt, entscheidet alles: wie schnell AES-256-GCM auf dem Kryptobeschleuniger läuft.** Liegt sie unter 3 MB/s, ist sie die neue Wand und der ganze Wechsel ist sinnlos. **Und sie ist nur am Gerät messbar** — es ist keines da. Deshalb lautet die ehrliche Antwort auf Thomas' Frage nicht „ja" oder „nein", sondern: **diese Entscheidung kann ohne Hardware nicht sauber fallen.**

9c-neu-2. „Was wäre der beste Chip?" — drei Antworten, je nach Preis

Thomas hat gefragt, was der beste Chip wäre. **Es gibt keinen einzelnen, und das liegt an einer Zange: USB High Speed, ein echter 4-Bit-Kartenanschluss, BLE und NFC-Tag-Emulation sind in keinem Chip zusammen zu haben.**

Der Befund, der die Chipwahl öffnet — und der meine Darstellung von gestern berichtigt: Ich hatte NFC für ein Ausschlusskriterium gehalten, weil nur Nordic eine Tag-Einheit mitbringt. **Das ist falsch.** Es gibt eigene NFC-Frontends mit Karten-Emulation, etwa den **ST25R3920B** (VFQFPN32), und bei denen läuft die ISO-14443-4- und ISO-7816-4-Schicht **im Rechenkern**. Genau diese Schicht haben wir **schon selbst geschrieben** (`docs/funk.md`: ISO-DEP 7.1–7.5, ISO 7816-4, Typ-4-Tag, CTAP 11.3, dazu 108 Millionen gefuzzte Rahmen). Sie bräuchte nur ein anderes Frontend statt der NFCT-Einheit. **NFC schließt fremde Rechenkerne also nicht aus.** Der PN7160 wäre der bequemere Weg, macht die Typ-4-Emulation aber nur für NDEF — für CTAP vermutlich zu eng (`ungeprueft`).

Weg	Chips	Durchsatz	Was er kostet
A: nRF54LM20A/B	1	**~3 MB/s** (Faktor 7,5)	kein SDMMC, kein echtes QSPI. Vorserie
B: ESP32-P4 / STM32H5 + NFC-Frontend + BLE	2–3	**~25 MB/s** (Faktor 60)	zwei bis drei Rechner mit Brücken dazwischen, neue Zephyr-Portierung, anderes Krypto-Backend, mehr Platz und Strom
C: nRF52840 bleibt	1	0,4 MB/s	nichts — lieferbar, Firmware läuft, Zulassung steht

Und eine Berichtigung zum QSPI-Nachteil, den ich zu schwer gewichtet habe. Der nRF54LM20 hat kein echtes QSPI, der 64-MB-NOR-Flash liefe dort mit rund 4 statt 16 MB/s. Das klingt nach Rückschritt, **ist aber keiner**: heute ist nicht der Flash der Engpass, sondern das USB mit 0,4 MB/s. Die 27 MB portabler Programme brauchen heute **68 Sekunden**; mit 4 MB/s Flash und 38 MB/s USB wären es **rund 7**. Also zehnmal besser statt schlechter. **Der Flash wird langsamer als er könnte, aber schneller als er heute genutzt wird** — das ist dasselbe Muster wie überall in diesem Projekt: zwei Zahlen, die dasselbe heißen und Verschiedenes messen.

Empfehlung: Weg A, sobald lieferbar. Begründung in dieser Reihenfolge:

1. **Ein Chip.** Für ein Gerät, dessen Zweck Sicherheit ist, sind zwei oder drei Rechner mit Brücken dazwischen die schlechtere Antwort — auch wenn Weg B achtmal schneller wäre. Mehr Teile heißt mehr Angriffsfläche, und der Entwurf oben begründet an mehreren Stellen, warum Einfachheit hier Sicherheit ist.
2. **512 KB RAM statt 256.** Das löst das Problem der 128 KB Merkle-Cache, ohne die der Zugriff auf Pfadknoten auf 0,14 MB/s einbricht.
3. **2 MB Flash statt 1.** RSA, ML-KEM, age, CXF, CSR und die CA-Verwaltung müssen alle hinein.
4. **Das Modul ist kleiner** als das heutige (11 × 8 × 1,8 gegen 15,5 × 10,5 × 2,05 mm), und Thomas' Hauptwunsch ist Kürze.
5. Faktor 7,5 genügt dem Zweck: 1 GB in 5,6 Minuten ist für Dokumente, Schlüssel und Notfallunterlagen reichlich.

Weg B bleibt die Antwort, falls jemand später einen echten Datenträger will — dann aber als anderes Gerät, nicht als Ausbaustufe dieses.

9c. Was damit endgültig nicht gebaut wird

Drei Dinge fallen aus der Anforderungsliste, und zwar begründet statt stillschweigend:

1. **Verschlüsselter Massenspeicher** — Durchsatz, siehe oben.
2. **M-von-N auf Schlüsselexport** — der Chip kann keinen Schlüsselexport (Abschnitt 3). Was bleibt, ist Shamir auf dem Dateischlüssel der Sicherung, und das ist bereits gebaut.
3. **Fingerabdrucksensor in Stufe 1** — kein beschaffbarer Sensor erfüllte alle vier Anforderungen. Die Stiftreihe J5 bleibt auf der Platine, damit es später geht; die Breite von 27 mm steht dafür.

10. Arbeitsschritte

Reihenfolge nach Thomas' Entscheidung: das Dringende zuerst, dokumentiert wird danach. Schritte 0 und 1 laufen also vor dem Marktvergleich, nicht neben ihm.

0. **Die Chip-Bestellnummer festnageln** (Abschnitte 7b und 7f). Gesucht ist **eine Bestellnummer**, keine Variantenbezeichnung, deren Konfiguration **RSA, EdDSA, Curve25519 und ECDH zusammen** anbietet. Quelle ist NXPs AN12973, nötigenfalls über einen Distributor belegt. **Die erwartete Antwort ist unangenehm:** die einzige in den Unterlagen belegte Maske (`0x27F2`, aus der FIPS-Politik) hat RSA an und EdDSA aus. **Findet sich keine Konfiguration mit beidem, lautet der Befund genau so** — und nicht „vermutlich geht es schon". Dann geht die Frage an Thomas zurück, mit JavaCard als Alternative. Bis die Nummer dasteht, ist jede Zusage zu SSH-Schlüsseln, Geldbörse und Server-Schlüsseleinigung `ungeprueft`. Thomas bestellt selbst; hier wird nur belegt.

Dazu, im selben Schritt: **wie viele RSA-4096-Schlüssel in den Nutzspeicher passen** — das hängt an Widerspruch 4 (50 oder 104 kB) und begrenzt, was an Kapazität zugesagt werden darf. Und die **Flash-Rechnung aus Bedingung 2** (Abschnitt 9b): passen RSA, ML-KEM, age, OpenPGP-Nachrichten, CXF, CSR und die CA-Verwaltung neben MCUboot in das Megabyte?

Der Massenspeicher-Durchsatz gehört ebenfalls hierher, weil er eine Zusage im Handbuch betrifft: die 400 kB/s sind aus Fremdmessungen übernommen (Abschnitt 9a) und für unseren Aufbau `ungeprueft`. Sobald Hardware da ist, wird gemessen, und das Handbuch bekommt die Wartezeit genannt — nicht die Rechnung, sondern die gemessene Sekundenzahl.

1. **Dann die acht Widersprüche aus Abschnitt 6 auflösen** — gemessen, nicht diskutiert. Für jeden: welche Stelle stimmt, woran man es festmacht, und die falsche Stelle berichtigen statt beide stehenzulassen. Der ARM-Bau und die Credential-Zahl zuerst, weil der Marktvergleich auf beiden ruht. Wer eine Begründung zurücknimmt, sagt ausdrücklich, ob der Befund mitfällt.

1. **Neue Datei** `docs/marktvergleich.md` mit allem aus den Abschnitten 1–5 **und 8**, jede fremde Fähigkeit mit Quelle (Yubico-Handbuch, Datenblatt, Herstellerdoku), jede eigene Zeile mit der Datei, die sie belegt. **Drei Spalten, nicht zwei: Fremdprodukt — unsere Zusage — unser Stand.** Aufbau: erst die vier harten Lücken, dann die weiteren, dann was aus dem HSM taugt, dann die Zertifizierungen, dann der eigene Vorsprung. Die Datei ist der Bezugspunkt für alles Weitere und wird **nicht** in andere Dokumente kopiert — Hausregel „eine Zahl nur an einer Stelle".
2. `docs/threat-model.md` **ergänzen** — vier Einträge:
   - der Entwurfsgegensatz aus Abschnitt 3: Schlüsselexport mit Wrapping gegen „verlässt den Chip nie", und dass der Chip es **nicht kann**
   - die PIN-Grenze aus Abschnitt 1 um den Hinweis, dass eine Eingabe am Gerät sie aufheben würde
   - **Post-Quanten als Grenze, die bisher nicht einmal benannt ist** (8d), samt der Regel „hybrid oder gar nicht" und der Feststellung, dass die Sicherungsdatei nicht betroffen ist
   - **CXF-Export** (8c): Zugangsdaten können das Gerät verlassen, mit welcher Freigabe, und dass die Anzeige das Ziel nennen muss
3. **Die Stufenliste oben ergänzen**, ohne sie umzubauen:
   - Stufe 3 (FIDO2): **CredentialManagement (0x0A) und Config (0x0D)** als eigene Punkte, dazu Enterprise Attestation und der Signaturzähler im Flash
   - Stufe 5 (Smartcard): **Yubico-HID-Transport** (davon hängen KeePassXC und VeraCrypt ab), statisches Passwort, PIV-Attestierung je Steckplatz, PIN-Regelwerk, eindeutige Seriennummer in der OpenPGP-AID
   - Stufe 6 (Tresor): **monotoner Zähler im Chip gegen das Rückdrehen**
   - Stufe 11 (Anzeige): **PIN-Eingabe am Gerät** als eigenen Prüfpunkt, mit der offenen Frage nach der Tasterzahl
   - Stufe 14 (Dongle/Server): YubiHSM Auth, Rechtebereiche, verkettetes Protokoll
   - Stufe 17 (Alltag): Selbstzerstörungs-PIN, Abstreitbarkeit, verschlüsselter Massenspeicher
   - Stufe 18: Zertifizierungen als benannter offener Punkt

Nicht aufgenommen, weil der Chip es nicht kann: M-von-N auf Schlüsselexport (Abschnitt 3). **Nicht aufgenommen, weil es eine Entscheidung ist, keine Lücke:** RSA. Beides gehört ins Bedrohungsmodell, nicht in eine Stufe — und RSA ist die Frage, die Thomas beantworten muss, bevor irgendwer PIV für Firmenumgebungen verspricht.

4. **Kapazitäten festlegen und dokumentieren** — wie viele Passkeys, PIV- Zertifikate, OATH-Einträge, Tresoreinträge das Gerät führt. Yubico nennt seine Zahlen, wir haben keine. Das folgt aus dem Nutzspeicher des Sicherheitschips und dem Flash — und setzt damit Widerspruch 4 und die RSA-Rechnung aus Schritt 0 voraus.
5. **Die zwei Firmware-Varianten als Baueinstellung anlegen** (Abschnitt 7f): eine Kconfig-Option, zwei Abbilder, Variante im Dateinamen **und** im Abbildkopf, zwei AAGUIDs vorgesehen, `tools/` und der reproduzierbare Bau je Variante. Das ist Vorarbeit und Dokumentation — die RSA-Umsetzung selbst ist eine eigene Stufe und wartet auf die Bestellnummer aus Schritt 0.
6. **Die Stufenliste um die Befunde aus Abschnitt 8 ergänzen** — das sind die größeren Lücken, und drei davon stehen oben als Zusage ohne Entsprechung:
   - **Stufe 5** (Smartcard): **CSR nach PKCS#10 auf dem Gerät** — das ist die Grundfunktion, ohne die kein Zertifikat zu einem Chip-Schlüssel entsteht. Dazu X.509-Erweiterungen (SAN, Key Usage, EKU)

- **Stufe 8** (Anbindungen): ein **Dateiwerkzeug** `osk encrypt` / `osk decrypt` mit **age** und OpenPGP-Nachrichten, **mit Blockverfahren** — ohne Streaming ist es bei der ersten großen Datei am Ende. Dazu `sshsig`, Git-Commit-Signatur und RFC-3161-Zeitstempel
- **Stufe 13 oder eigene Stufe**: **CXF-Import und -Export** für Passkeys, Passwörter, TOTP und SSH-Schlüssel, mit PIN, Tastendruck und einer Anzeige, die sagt **wohin**
- **Stufe 17** (Alltag): Importwege aus KeePass, CSV und Browsern; Leak-Prüfung über k-Anonymität; Stärkeprüfung. **Der verschlüsselte Massenspeicher kommt nicht hinein** — gestrichen nach Abschnitt 9b
- **Stufe 6** zusätzlich: `osk_se050_counter_read()` und `counter_raise()` für den Rückdrehschutz. Der Chip hat die Zähler (9a) — und die beiden Aufrufe blockieren heute schon die Paketprüfung des Bootladers
- **Neue Stufe für die eigene CA**: Kette, Zwischenzertifikate, **Widerrufsweg (CRL oder OCSP)**, Ablaufwarnung. Eine CA ohne Widerruf wird nicht gebaut — das ist keine Ausbaustufe, sondern eine Bedingung
- **Querschnitt, keine Stufe: hybrides ML-KEM mit X25519** für Stick-zu-Stick, Server-zu-Server und CXP. Mit der Regel aus 8e: hybrid oder gar nicht, ein reiner PQC-Pfad ist ein Rückschritt

7. **JavaCard prüfen lassen** (Abschnitt 7c und 7g): ein eigener Strang klärt, ob ein JCOP-Teil ohne Geschäftskonto beschaffbar ist, was die Werkzeugkette kostet (Oracle JDK, JCOP-Plugin, GPShell), wie die Hausregel „kein Fremdcode" dazu steht — das Hersteller-SDK bliebe Werkzeug wie das nRF-SDK, aber das gehört ausgesprochen — und **was von den 12 000 Zeilen C bliebe, wenn die Sicherheitslogik in den Chip wandert**. Ergebnis ist eine Entscheidungsvorlage mit Zahlen, kein Umbau und keine Bestellung.

11. Prüfung

Schritt 0 wird gemessen, der Rest gegen die Unterlagen geprüft. Beim ARM-Bau heißt messen: einen Bau anstoßen und die Ausgabe zeigen, nicht eine der beiden Dokumentstellen für glaubwürdiger erklären. Bei der Credential-Zahl und beim Chipspeicher heißt es: die Datenblattstelle nennen, die es entscheidet.

Für die Dokumente gilt:

- Jede fremde Fähigkeit in `marktvergleich.md` trägt eine Quelle, und die Quelle ist nachlesbar (Handbuchseite, Datenblatt, Herstellerdoku). Was nur aus einer Zusammenfassung Dritter stammt, wird als `ungeprueft` gekennzeichnet — bei der OATH-Kapazität von 32 und den Nitrokey-Passkey-Zahlen ist das der Fall.
- Jede Zeile „Stand bei uns" wird gegen die Unterlagen belegt, nicht gegen Erinnerung. Wo der Entwurf etwas zusagt, steht die Datei dabei; wo er schweigt, heißt es „nicht zugesagt" und nicht „fehlt".
- Gegenprobe gegen die vier Stufen von „geprüft": keine Fähigkeit dieses Entwurfs wird in der Vergleichstabelle als vorhanden geführt, nur weil sie geplant ist. Der Vergleich ist **Zusage gegen Produkt**, und das muss dastehen, sonst liest ihn jemand als Leistungsvergleich.